

REPORT

파이토치를 활용한 차량 5부제 자동 단속 시스템



이름	오 진 우	학과	인공지능학과
학번	22012193	제출일자	2026-06-21
과목명	딥러닝실습	교수명	구 영 현

목 차

1. 프로젝트 개요	4~10
1.1 기획 배경 및 필요성	4
1.2 프로젝트 의의 및 활용 방안	5
1.3 프로젝트 초기 목표	6
1.4 초기 파이프라인 구현의 한계	7
1.5 최종 시스템 구성 방향	8~9
1.6 최종 시스템 파이프라인	10
2. 데이터셋 구성 및 전처리	11~18
2.1 YOLOv8 데이터셋	11~13
2.1.1 데이터 출처 및 선정 기준	
2.1.2 데이터 클래스 구성	
2.1.3 JSON 라벨 구조	
2.1.4 YOLO 형식 변환	
2.1.5 학습·검증·테스트 데이터 분할	
2.1.6 데이터셋 특징 및 한계	
2.2 ResNet18 데이터셋	14~18
2.2.1 데이터 출처 및 선정 기준	
2.2.2 데이터 클래스 구성	
2.2.3 일반 번호판 수집	
2.2.4 친환경 번호판 수집	
2.2.5 친환경 번호판 데이터 증강	
2.2.6 학습·검증·테스트 데이터 분할	
2.2.7 입력 이미지 전처리	
3. 모델 설계 및 학습	19~29
3.1 YOLOv8 모델 학습	19~22
3.1.1 모델 선정 이유	
3.1.2 모델 구조	
3.1.3 학습 데이터 설정	
3.1.4 학습 파라미터	
3.1.5 학습 코드	
3.2 ResNet18 모델 학습	23~28
3.2.1 모델 선정 이유	
3.2.2 모델 구조	
3.2.3 학습 데이터 설정	
3.2.4 학습 파라미터	
3.2.5 학습 코드	
3.3 EasyOCR 모델	29

4. 모델 성능 평가	30~40
4.1 YOLOv8 모델 성능 평가	30~35
4.1.1 최종 성능 지표	
4.1.2 학습 곡선 분석	
4.1.3 Confusion Matrix 분석	
4.1.4 Precision-Recall Curve 분석	
4.1.5 F1-Confidence Curve 분석	
4.1.6 YOLOv8 모델의 한계점	
4.2 ResNet18 모델 성능 평가	36~40
4.2.1 최종 성능 지표	
4.2.2 학습 곡선 분석	
4.2.3 Confusion Matrix 분석	
4.2.4 ROC Curve 분석	
4.2.5 ResNet18 모델의 한계점	
5. 시스템 구현 및 배포	41~47
5.1 시스템 구성 및 개발 환경	41
5.2 모드 1: CCTV 이미지 분석	42
5.3 모드 2: 번호판 이미지 분석	43
5.4 Streamlit 시스템 구현	44~45
5.5 Hugging Face Spaces 배포	46
5.6 시스템 구현 결과 및 한계	47
6. 결론 및 요약	48
7. 참고 문헌	49

1. 프로젝트 개요

1.1 기획 배경 및 필요성

최근 국제 정세 불안과 유가 변동성 증가로 인해 차량 운행량을 효율적으로 관리하기 위한 정책적 필요성이 커지고 있다. 차량 5부제는 번호판 끝자리를 기준으로 특정 요일의 차량 운행을 제한하는 제도로, 에너지 절약, 교통량 감소, 주차 수요 완화, 미세먼지 저감 등의 효과를 기대할 수 있다. 이에 따라 일부 기관과 기업에서는 자율적으로 차량 5부제를 시행하고 있으며, 상황에 따라 차량 2부제와 같은 운행 제한 정책이 한시적으로 적용되는 사례도 나타나고 있다.

그러나 현재 차량 5부제 단속은 고정식 CCTV, 단속 카메라, 관리 인력의 육안 확인 방식이 혼용된 수동 중심 구조로 이루어져 있다. 이러한 방식은 차량이 많은 출입구나 주차장 환경에서 모든 차량을 빠르게 확인하기 어렵고, 단속 인력의 판단에 따라 정확도 편차가 발생할 수 있다. 또한 지속적인 인력 투입이 필요하므로 운영 비용이 증가하고, 실시간 대응에도 한계가 존재한다.

이에 본 프로젝트에서는 딥러닝 기반 차량 및 번호판 인식 기술을 활용하여 차량 5부제 위반 여부를 자동으로 판정하는 시스템을 구축하고자 한다. YOLOv8을 이용해 차량과 번호판 영역을 탐지하고, ResNet18로 친환경 번호판 여부를 분류한 뒤, 일반 번호판에 대해 EasyOCR로 끝자리 숫자를 추출하여 요일별 제한 번호와 비교한다. 이를 통해 수동 단속 방식의 한계를 보완하고, 향후 주차장 관리 시스템이나 지능형 교통 관리 시스템으로 확장 가능한 자동 단속 기반을 마련하고자 한다.

1.2 프로젝트 의의 및 활용 방안

본 프로젝트의 핵심 의의는 단순히 번호판 문자를 인식하는 데 그치지 않고, 실제 차량 5부제 단속 절차를 고려하여 차량 및 번호판 탐지, 친환경 번호판 분류, 끝자리 숫자 추출, 요일별 제한 번호 판정 과정을 하나의 시스템 흐름으로 구성했다는 점이다. 특히 친환경 번호판으로 분류된 차량은 OCR을 수행하지 않고 즉시 단속 면제로 처리하고, 일반 번호판에 대해서만 끝자리 숫자를 추출하여 해당 요일의 제한 번호와 비교하도록 설계하였다. 이를 통해 실제 단속 과정에서 필요한 예외 처리와 판정 기준을 반영한 자동화 시스템을 제안한다.

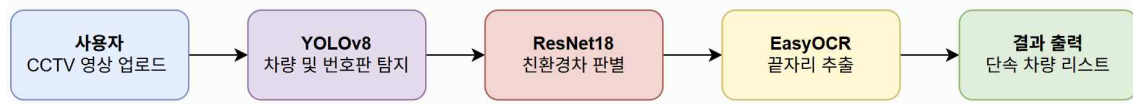
또한 본 프로젝트는 YOLOv8, ResNet18, EasyOCR을 각각 활용하여 객체 탐지, 이미지 분류, 문자 인식 기능을 분리 구현하고, 이를 Streamlit 기반 웹 인터페이스로 통합했다는 점에서도 의의가 있다. 사용자는 별도의 복잡한 실행 과정 없이 이미지를 업로드하여 차량 및 번호판 탐지 결과와 5부제 판정 결과를 확인할 수 있으며, 이는 기존의 수동 확인 방식보다 일관된 기준으로 결과를 제공할 수 있다는 장점이 있다.

단기적으로는 기업·기관 주차장, 학교, 공공시설 등에서 차량 5부제 자율 단속을 보조하는 시스템으로 활용할 수 있다. 또한 Hugging Face Spaces에 배포된 웹 데모를 통해 별도 설치 없이 시스템의 동작 과정을 확인할 수 있으므로, 딥러닝 기반 차량 단속 시스템의 기술 검증 및 시연 자료로 활용 가능하다.

중장기적으로는 주차장 입출차 관리 시스템이나 차량 등록 데이터베이스와 연동하여 차량별 단속 이력 저장, 반복 위반 차량 확인, 관리자 조회 기능으로 확장할 수 있다. 더 나아가 요일별 제한 번호 테이블을 수정하면 차량 5부제뿐만 아니라 홀짝제, 요일제, 특정 시간대 운행 제한 등 다른 차량 운행 제한 정책에도 적용할 수 있으며, 향후 지능형 교통 관리 시스템으로 발전시킬 수 있는 기반을 마련한다.

1.3 프로젝트 초기 목표

본 프로젝트의 초기 목표는 CCTV 이미지로부터 차량 및 번호판을 자동으로 탐지하고, 딥러닝 기반의 3단계 파이프라인(YOLOv8 → ResNet18 → EasyOCR)을 통해 차량 5부제 위반 여부를 자동 판별하는 시스템 구축을 목표로 하였다.



위의 그림과 같이 사용자가 Streamlit 기반 시스템에 CCTV 이미지를 업로드하면, 각 이미지는 순차적으로 3단계 모델 추론 과정을 거쳐 최종 단속 여부가 판정된다.

1단계 (YOLOv8): 입력 이미지에서 차량과 번호판의 위치를 bounding box 형태로 검출하고, 탐지된 번호판 영역을 crop하여 다음 단계의 입력으로 전달한다.

2단계 (ResNet18): crop된 번호판 이미지를 이진 분류 모델에 입력하여 친환경 차량 여부를 판별한다. 친환경 번호판으로 판정된 경우 단속 대상에서 즉시 제외한다.

3단계 (EasyOCR): 일반 차량으로 분류된 번호판에 대해 OCR을 적용하여 끝자리 숫자를 추출하고, 해당 요일의 운행 제한 번호와 비교하여 최종 위반 여부를 판정한다.



3단계에 거친 추론 과정 후 최종 판정 결과는 위의 그림과 같이 정상 통과, 단속 대상, 친환경 면제로 구분되어 화면에 출력되도록 설계하였다.

1.4 파이프라인 구현의 한계

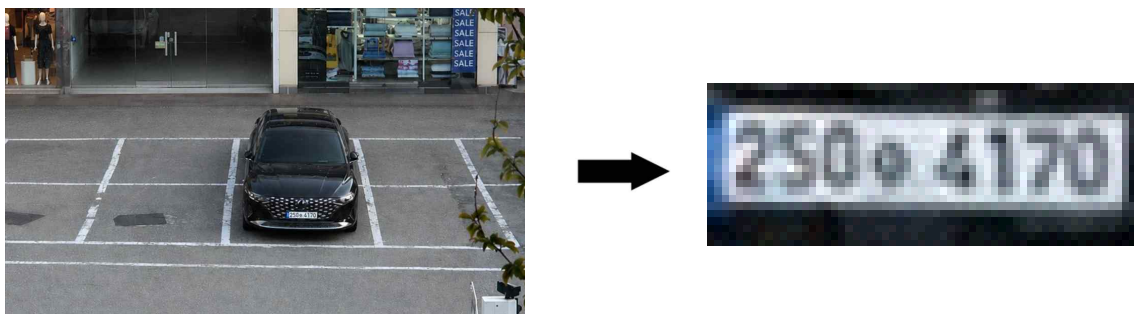
위와 같이 CCTV 영상으로 전체 파이프라인을 구동하는 것을 목표로 하였으나, 구현 과정에서 다음과 같은 세 가지 한계에 직면하였다.

1) 공개 데이터셋의 번호판 모자이크로 인한 검증 제한

국내에 공개된 CCTV 차량 데이터셋은 개인정보 보호를 위해 번호판 영역이 전부 모자이크 처리된 형태로 제공된다. YOLOv8은 모자이크 처리된 번호판 영역도 객체로 탐지할 수 있지만, 이후 단계인 ResNet18의 친환경 분류와 EasyOCR의 문자 인식은 실제 번호판 이미지가 필요하므로 해당 데이터셋만으로는 전체 파이프라인의 end-to-end 검증이 불가능하였다.

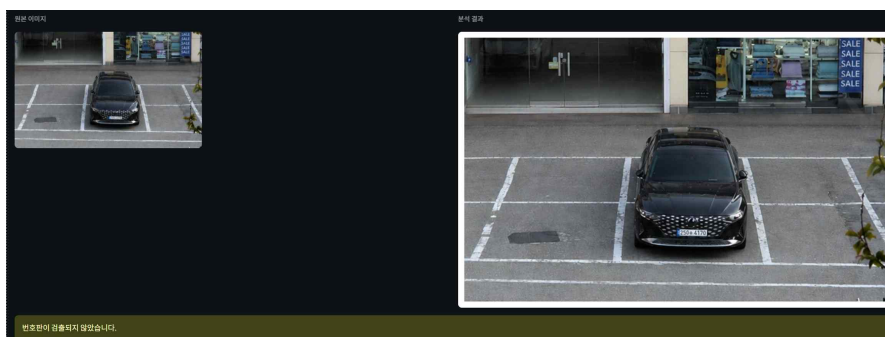
2) 원거리 촬영에 따른 번호판 해상도 저하

첫 번째 문제를 보완하기 위해 CCTV와 유사한 하향식 각도로 차량을 직접 촬영하여 테스트를 시도하였다. 그러나 CCTV의 특성상 촬영 거리가 멀어 YOLOv8이 탐지한 번호판 crop 이미지의 해상도가 지나치게 낮았으며, 이로 인해 EasyOCR을 통한 번호판 문자 인식이 정상적으로 이루어지지 않았다.



3) 촬영 거리 변화에 따른 차량 검출 성능 저하

OCR 인식률을 높이기 위해 차량을 보다 가까이서 촬영하는 방식을 시도하였으나, 이 경우 학습 데이터의 촬영 각도 및 거리와 크게 달라져 YOLOv8의 차량 검출 자체가 실패하는 문제가 발생하였다.

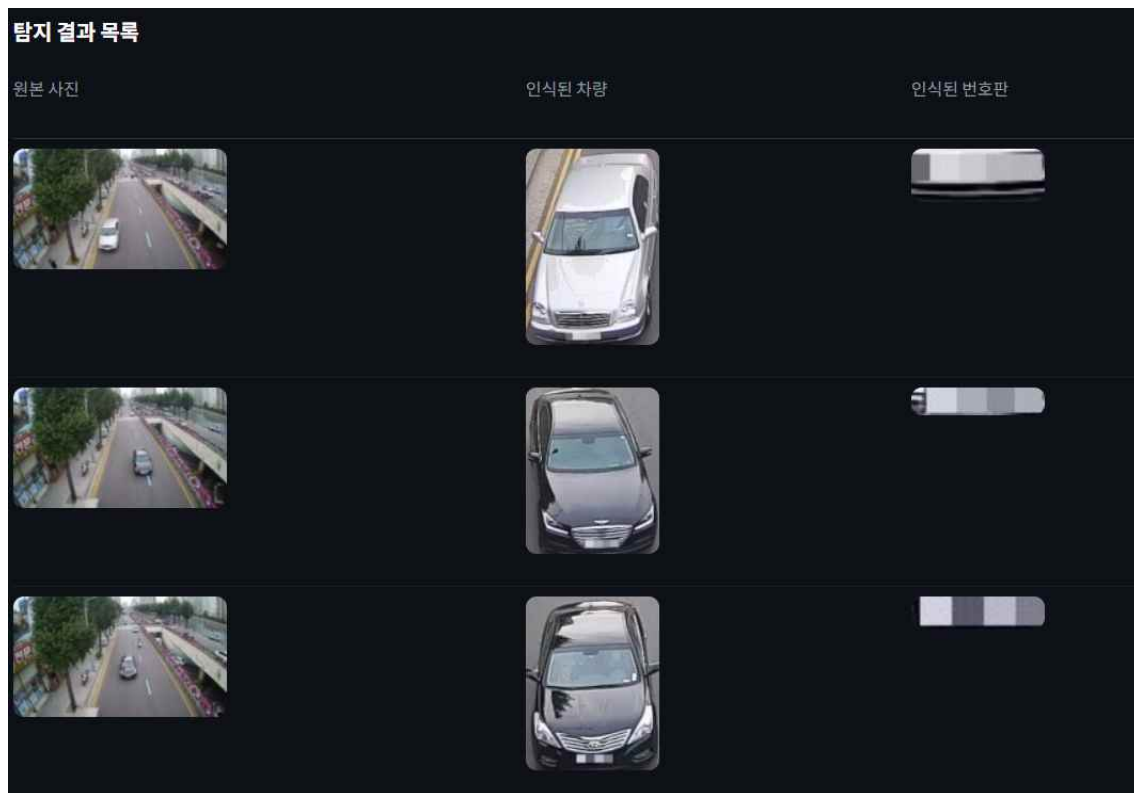


1.5 최종 시스템 구성 방향

최종적으로, CCTV 이미지 한 장으로 전체 파이프라인을 구동하는 방식은 현실적으로 검증이 어려웠다. 이에 본 프로젝트는 전체 파이프라인을 두 가지 모드로 분리하여 각 단계를 독립적으로 검증하는 방식으로 시스템 구성 방향을 전환하였다.

모드1 (CCTV 이미지 분석)

CCTV 이미지를 업로드하면 YOLOv8이 차량과 번호판 위치를 탐지하고, 검출된 차량 및 번호판 영역을 crop 이미지 형태로 반환한다. 모자이크 처리가 포함된 공개 데이터셋에서도 탐지 자체는 가능하므로, 본 모드를 통해 YOLOv8의 탐지 성능을 독립적으로 검증한다.



위의 그림과 같이 CCTV 이미지를 업로드하면 YOLOv8이 차량과 번호판 영역을 각각 탐지하고, 검출된 영역을 crop하여 그 결과를 확인한다. crop된 번호판은 AI hub 데이터셋 특성상 모자이크 처리된 형태로 반환되며, 이는 1.4절에서 기술한 데이터셋 한계에 해당한다.

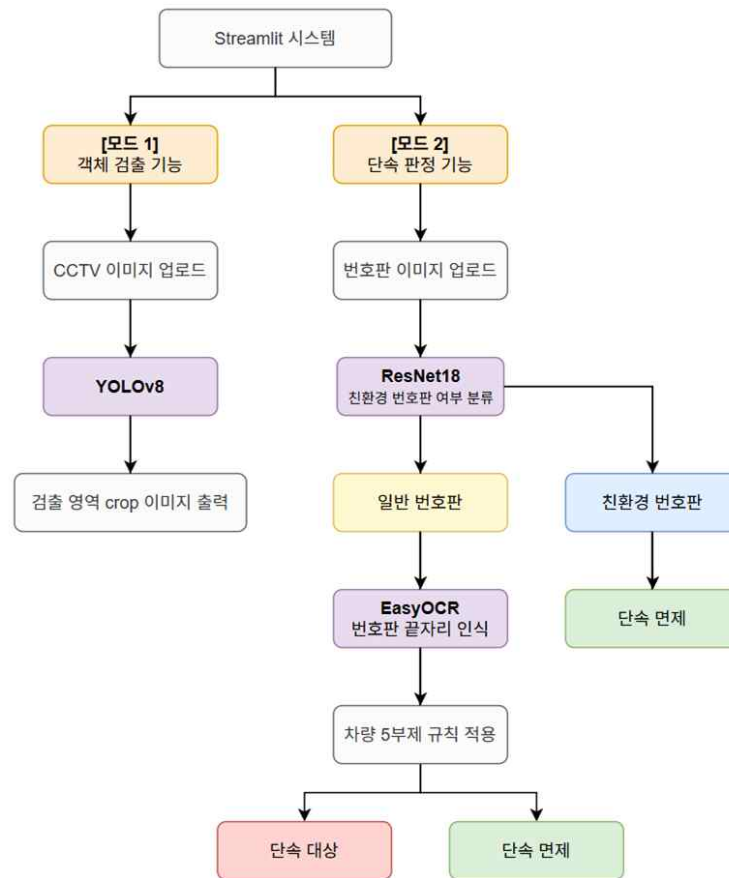
모드2 (번호판 이미지 분석)

번호판 이미지를 직접 업로드하면 ResNet18로 친환경 여부를 분류하고, 일반 번호판으로 판정된 경우 EasyOCR을 통해 끝자리 숫자를 추출하여 5부제 위반 여부를 판정한다. 본 모드에서는 시중에 공개된 번호판 데이터셋을 YOLOv8이 crop한 번호판으로 가정하고 활용하였으며, 이를 통해 ResNet18과 EasyOCR 단계의 성능을 독립적으로 검증한다.

단속 결과 목록			
캡처	종류	끝자리	판정
	친환경	7	면제
	친환경	6	면제
	친환경	5	면제
	일반	4	정상
	일반	9	정상
	일반	0	위반

위의 그림은 모드 2의 실제 단속 판정 결과를 나타낸다. 번호판 이미지를 업로드하면 ResNet18이 친환경 여부를 분류하고, 일반 번호판으로 판정된 경우 EasyOCR이 끝자리 숫자를 추출하여 해당 요일의 운행 제한 번호와 비교한 뒤 최종 판정 결과를 출력한다.

1.6 최종 시스템 파이프라인



위의 그림은 최종 시스템의 전체 파이프라인을 나타낸다. Streamlit 기반 웹 인터페이스는 객체 검출 기능과 단속 판정 기능의 두 가지 모드로 구성되며, 각 모드는 독립적으로 동작한다.

모드 1에서는 CCTV 이미지를 입력받아 YOLOv8로 차량과 번호판을 검출하고 해당 영역을 crop 이미지로 출력한다. 모드 2에서는 번호판 이미지를 입력받아 ResNet18로 친환경 여부를 분류한다. 일반 번호판은 EasyOCR로 끝자리 숫자를 인식한 후 차량 5부제 규칙을 적용하며, 친환경 번호판은 즉시 단속 면제로 처리한다.

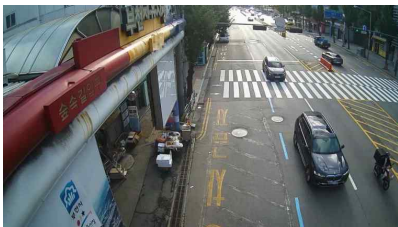
향후 고해상도 CCTV 영상이나 번호판이 모자이크 처리되지 않은 실제 데이터셋이 확보된다면, 현재 분리된 두 모드를 하나의 연속된 파이프라인으로 통합하여 완전한 end-to-end 차량 5부제 자동 단속 시스템으로 발전시킬 수 있다.

2. 데이터셋 구성 및 전처리

2.1 YOLOv8 데이터셋

2.1.1 데이터 출처 및 선정 기준

차량 및 번호판 탐지 학습을 위해 AI Hub에서 제공하는 자동차 차종/연식/번호판 인식용 이미지 데이터셋(AI Hub #172)을 사용하였다. 해당 데이터셋은 실제 CCTV 환경에서 수집된 차량 이미지와 차량 및 번호판 정보가 포함된 JSON 라벨 파일로 구성되어 있다. 본 프로젝트에서는 사용 가능한 JSON 라벨 파일 5,500개를 기준으로, 이에 대응하는 CCTV 이미지 5,500장을 함께 활용하였다.



CCTV 이미지 5500장

```
{ "imagePath": "자유시장앞_고정1-20200908-075958-001_2110.jpg", "car":  
{ "bbox": [[1400.8924560546875, 512.951171875],  
[1668.80224609375, 869.3295288085938]], "imagePath": "SUV/BMW/SUV_X7-  
1.jpg", "attributes": { "brand": "BMW", "color": "검은  
색", "model": "SUV_X7", "year": "2019-2020" }, "plate": { "bbox":  
[[1564.0819091796875, 843.5394287109375],  
[1615.64501953125, 856.5350341796875]] }, "id": "3b162f48-890d-458f-9943-  
b9286b6b533b", "videoName": "자유시장앞_고정1-20200908-075958-  
001.mp4", "frameNo": 2110, "weather": "맑음,비" }
```

json 라벨 파일 5500개

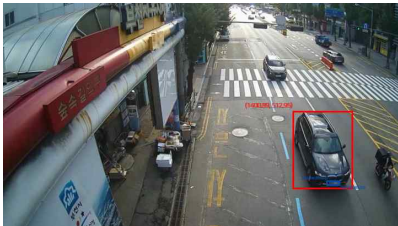
2.1.2 데이터 클래스 구성

클래스 번호	클래스 이름	설명
0	car	차량 영역
1	plate	번호판 영역

YOLOv8 학습을 위해 객체 클래스를 차량과 번호판의 두 가지로 구성하였다. 차량은 class 0, 번호판은 class 1로 지정하였으며, 하나의 CCTV 이미지에서 두 객체를 동시에 탐지하도록 학습하였다. 이는 번호판만 단독으로 검출할 경우 도로 표지판이나 간판처럼 형태가 유사한 객체를 오탐지할 가능성이 있기 때문이다.

2.1.3 JSON 라벨 구조

AI Hub에서 제공하는 원본 라벨은 JSON 형식으로 구성되어 있으며, 각 이미지의 크기 정보와 차량 및 번호판 객체의 bounding box 좌표가 포함되어 있다. Bounding box는 객체의 좌측 상단과 우측 하단 위치를 기준으로 한 절대 픽셀 좌표로 저장되어 있으며, 하나의 이미지에 여러 객체가 존재하는 경우 각 객체의 클래스와 좌표 정보가 함께 기록된다.



CCTV 이미지 bbox 시각화

```
{
  "imagePath": "자유시장앞_고정1-20200908-075958-001_2110.jpg",
  "car": {
    "bbox": [[1400, 8924560546875, 512.951171875],
            [1668, 80224609375, 869.3295288085938]],
    "imagePath": "SUV/BMW/SUV_X7-1.jpg",
    "attributes": {
      "brand": "BMW",
      "color": "검은색",
      "model": "SUV_X7",
      "year": "2019-2020"
    },
    "plate": {
      "bbox": [[1564, 0819091796875, 843.5394287109375],
              [1615, 64501953125, 856.5350341796875]]
    },
    "id": "3b162f48-890d-458f-9943-b9286b6b533b",
    "videoName": "자유시장앞_고정1-20200908-075958-001.mp4",
    "frameNo": 2110,
    "weather": "맑음,비"
  }
}
```

json 라벨 파일

2.1.4 YOLO 형식 변환

YOLOv8은 이미지별 객체 정보를 TXT 형식의 라벨 파일로 입력받는다. 따라서 JSON 라벨에서 차량과 번호판의 클래스 및 bounding box 좌표를 추출하여 이미지와 동일한 이름의 TXT 파일로 변환하였다. 각 라벨은 다음과 같은 형식으로 저장했다.

class_id	x_center	y_center	width	height
----------	----------	----------	-------	--------

x_center와 y_center는 bounding box 중심점의 좌표이며, width와 height는 객체 영역의 가로와 세로 크기를 의미한다. 이 값의 계산 과정은 아래와 같다.

```
x_center = ((x_min + x_max) / 2) / image_width
y_center = ((y_min + y_max) / 2) / image_height
width = (x_max - x_min) / image_width
height = (y_max - y_min) / image_height
```

따라서 bounding box의 좌측 상단 좌표((x_{min}, y_{min}))와 우측 하단 좌표((x_{max}, y_{max}))를 이용하여 중심점(x_center, y_center)과 크기를 계산하였다.

이후 각 값을 이미지의 전체 가로 및 세로 크기로 나누어 0~1 범위로 정규화하였다. 이는 이미지마다 해상도가 서로 다르더라도 객체의 위치와 크기를 동일한 비율 기준으로 표현하여 YOLOv8이 일관된 형식으로 학습할 수 있도록 하기 위함이다. 최종적으로 변환된 TXT 라벨 파일의 형태는 아래의 그림과 같다.

0	0.804	0.639	0.139	0.330	← 차량 (class 0)
1	0.821	0.778	0.027	0.012	← 번호판 (class 1)

2.1.5 학습·검증·테스트 분할

전체 5,500장의 CCTV 이미지는 학습용 80%, 검증용 10%, 테스트용 10%의 비율로 분할하였다.

구분	비율	이미지 수	사용 목적
Train	80%	4,400장	모델 가중치 학습
Validation	10%	550장	학습 중 성능 확인 및 과적합 여부 판단
Test	10%	550장	최종 탐지 성능 평가

이에 따라 Train 데이터는 4,400장, Validation 데이터는 550장, Test 데이터는 550장으로 구성하였다. Train 데이터는 모델의 가중치 학습에 사용하고, Validation 데이터는 학습 과정에서 성능 변화와 과적합 여부를 확인하는 데 활용하였다. Test 데이터는 학습에 사용하지 않고 최종 객체 검출 성능을 평가하는 용도로 사용하였다.

2.1.6 데이터셋 특징 및 한계

해당 데이터셋은 실제 CCTV 환경에서 촬영된 차량 이미지로 구성되어 있어 원거리 촬영, 하향식 시점, 다양한 차량 배치 등 실제 감시 환경의 특징을 반영한다는 장점이 있다. 또한 차량과 번호판의 bounding box 라벨이 함께 제공되어 두 객체를 동시에 학습할 수 있다.

그러나 개인정보 보호를 위해 번호판 영역이 모자이크로 처리되어 있어 번호판 위치 검출은 가능하지만, 앞서 설명한 내용과 같이 번호판 색상 분류나 문자 인식에는 활용하기 어렵다는 한계가 있다. 또한 CCTV 촬영 특성상 번호판이 이미지에서 차지하는 영역이 작아 소형 객체 검출 난도가 높다는 특징이 있다.

또한 해당 데이터셋에 포함된 차량 종류는 다음과 같이 정리할 수 있다.

기아	현대	BMW
SUV_니로_EV	SUV_맥스크루즈	SUV_X1
SUV_니로	SUV_베뉴	SUV_X3
SUV_모하비	SUV_베라크루즈	SUV_X4
SUV_셀토스	SUV_싼타페	SUV_X5
SUV_스토닉	SUV_코나	SUV_X6
SUV_스포티지	SUV_테라칸	SUV_X7
SUV_쏘렌토	SUV_투싼	-
SUV_쏘울	SUV_팔리세이드	-
SUV_카렌스	-	-

따라서 학습 데이터에 포함되지 않은 트럭, 버스, 구급차와 같이 형태가 다른 차량에 대해서는 검출 성능이 저하되거나 검출에 실패할 수 있다.

2.2 ResNet18 데이터셋

2.2.1 데이터 출처 및 선정 기준

ResNet18 학습에는 친환경 번호판과 일반 번호판을 이진 분류하기 위한 번호판 이미지를 사용하였다. 번호판은 YOLOv8 학습에 활용한 AI Hub #172 데이터셋에서 제공된 번호판 이미지 80,000장을 활용하였다.



일반 번호판 사진



친환경 번호판 사진

2.2.2 데이터 클래스 구성

ResNet18 학습에 사용된 데이터셋은 일반 번호판(normal)과 친환경 번호판(eco) 두 클래스로 구성하였다. 클래스 인덱스는 eco가 0, normal이 1로 할당된다.

클래스 번호	클래스 이름	설명
0	eco (친환경 번호판)	파란색 번호판
1	normal (일반 번호판)	흰색/노란색/초록색 번호판

2.2.3 일반 번호판 수집

일반 번호판 이미지는 아래의 코드를 활용해 80,000장의 번호판 이미지 중 500장을 무작위로 추출하여 구성하였다. 이후 파란색 배경이 아닌 일반 번호판 이미지만 선별하였다.

```
def split_and_copy(src_dir, cls, n_sample=None):
    files = [f for f in os.listdir(src_dir)
              if f.lower().endswith((".jpg", ".png", ".jpeg"))]
    if n_sample:
        files = random.sample(files, n_sample) # 500장 랜덤 샘플링
    train, temp = train_test_split(files, test_size=0.2, random_state=42)
    val, test = train_test_split(temp, test_size=0.5, random_state=42)
    for split, file_list in [("train", train), ("val", val), ("test", test)]:
        for f in file_list:
            shutil.copy(os.path.join(src_dir, f),
                        os.path.join(DATASET_DIR, split, cls, f))

split_and_copy(NORMAL_DIR, "normal", n_sample=500)
```


2.2.4 친환경 번호판 수집

친환경 번호판 이미지는 AI Hub의 약 8만 장 번호판 이미지 중 극히 일부만 포함되어 있어 전체 데이터를 육안으로 확인하는 데 어려움이 있었다. 이에 아래의 코드를 이용하여 HSV 색상 필터를 적용하고, 친환경 번호판의 특징인 파란색 영역이 일정 비율 이상 포함된 이미지를 1차로 선별하였다.

```
import cv2
import numpy as np
import os
import shutil

SRC_DIR = r"C:/PyTorch Project/ResNet18/번호판 이미지 80,000장"
SAVE_DIR = r"C:/PyTorch Project/ResNet18/ECO 번호판"
os.makedirs(SAVE_DIR, exist_ok=True)

def is_eco_plate(image_path):
    img = cv2.imread(image_path, dtype=np.uint8)
    if img is None:
        return False

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    # 파란색 계열 HSV 범위
    lower_blue = np.array([100, 80, 80])
    upper_blue = np.array([130, 255, 255])

    mask = cv2.inRange(hsv, lower_blue, upper_blue)
    ratio = np.count_nonzero(mask) / mask.size # 파란색 픽셀 비율

    return ratio > 0.2 # 20% 이상이면 친환경 번호판으로 판단

files = [f for f in os.listdir(SRC_DIR)
          if f.lower().endswith((".jpg", ".png", ".jpeg"))]
count = 0

for f in files:
    path = os.path.join(SRC_DIR, f)
    if is_eco_plate(path):
        shutil.copy(path, os.path.join(SAVE_DIR, f))
        count += 1
    if count % 50 == 0:
        print(f"수집 중: {count}장")

print(f"\n친환경 번호판 수집 완료: {count}장 → {SAVE_DIR}")
```

1차 HSV 필터링 결과 총 10,512장의 이미지가 선별되었다. 이후 선별된 이미지를 육안으로 검토하여 친환경 번호판이 아닌 이미지를 제거한 결과, 총 107장의 친환경 번호판 이미지를 확보하였다.

추가적인 데이터 확보를 위해 웹 크롤링을 수행하였다. 크롤링 과정에서는 번호판 식별이 가능하고, 친환경 번호판의 특징인 파란색 배경이 명확하게 나타나는 이미지만 수집 대상으로 선정하였다.

친환경 번호판 이미지를 추가로 수집하기 위해 사용한 웹 크롤링 코드는 다음과 같다.

```
import os
import time
import requests
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

SAVE_DIR = r"C:/PyTorch Project/ResNet18/ECO 번호판"
TARGET = 200
os.makedirs(SAVE_DIR, exist_ok=True)

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
img_count = len(os.listdir(SAVE_DIR)) # 기존 파일 이어서 저장

queries = [
    "친환경 번호판",
    "전기차 번호판 파란색",
    "EV 번호판",
    "전기차 번호판",
    "파란 번호판 전기차",
    "친환경차 번호판 클로즈업",
]

for query in queries:
    if img_count >= TARGET:
        break

    print(f"\n검색어: {query}")
    driver.get(f"https://search.naver.com/search.naver?where=image&query={query}")
    time.sleep(2)

    for _ in range(10):
        driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
        time.sleep(1)

    imgs = driver.find_elements(By.CSS_SELECTOR, "img[class*='image']")
    if not imgs:
        imgs = driver.find_elements(By.TAG_NAME, "img")

    print(f" 이미지 {len(imgs)}개 발견")

    for img in imgs:
        if img_count >= TARGET:
            break
        src = img.get_attribute("src") or img.get_attribute("data-src")
        if not src or not src.startswith("http"):
            continue
        if any(x in src for x in ["logo", "icon", "btn"]):
            continue
        try:
            response = requests.get(src, timeout=5)
            if response.status_code == 200 and len(response.content) > 5000:
                path = os.path.join(SAVE_DIR, f"crawl_{img_count:04d}.jpg")
                with open(path, "wb") as f:
                    f.write(response.content)
                img_count += 1
                print(f" 저장 ({img_count}/{TARGET})")
        except:
            continue

    driver.quit()
    print(f"\n크롤링 완료 | 총 {img_count}장 수집")
```

크롤링 완료 후 친환경 번호판이 아닌 이미지(일반 번호판, 관련 없는 사진 등)는 수동 검수를 통해 제거하였다. 그 결과 21장의 이미지를 추가로 확보하여 최종적으로 총 128장의 친환경 번호판 데이터셋을 구성하였다.

2.2.5 친환경 번호판 데이터 증강

친환경 번호판은 원본 이미지가 128장으로 일반 번호판 500장에 비해 수가 적어 클래스 불균형이 발생할 수 있었다. 이를 보완하기 위해 RandomRotation($\pm 15^\circ$), RandomPerspective, RandomAffine(translate) 기법을 적용하여 친환경 번호판 이미지를 증강하였다.



번호판 이미지 원본



데이터 증강을 적용한 번호판 이미지

적용한 증강은 번호판의 회전, 원근 왜곡, 위치 이동 등 실제 촬영 환경에서 발생할 수 있는 변화를 반영하기 위한 기하학적 변환이다. 반면 친환경 번호판 분류의 핵심 특징인 파란색 배경을 유지하기 위해 ColorJitter, 좌우 반전, Cutout 등 색상이나 번호판 구조를 크게 변화시킬 수 있는 기법은 제외하였다. 증강 결과 친환경 번호판 데이터를 총 512장으로 확장하였다.

친환경 번호판 이미지 증강을 위해 사용한 코드는 다음과 같다.

```
from torchvision import transforms
from PIL import Image
import os

ECO_DIR = r"C:/PyTorch Project/ResNet18/ECO 번호판"
ECO_AUG_DIR = r"C:/PyTorch Project/ResNet18/ECO 번호판 증강"

os.makedirs(ECO_AUG_DIR, exist_ok=True)

augments = [
    transforms.RandomRotation(15),
    transforms.RandomPerspective(distortion_scale=0.2, p=0.5),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
]

files = [f for f in os.listdir(ECO_DIR) if f.lower().endswith((".jpg", ".png", ".jpeg"))]

count = 0
for f in files:
    img = Image.open(os.path.join(ECO_DIR, f)).convert("RGB")
    for aug in augments:
        result = aug(img)
        result.save(os.path.join(ECO_AUG_DIR, f"aug_{count:04d}.jpg"))
        count += 1

print(f"증강 완료: {count}장 → {ECO_AUG_DIR}")
```

2.2.6 학습·검증·테스트 분할

구성된 번호판 이미지는 아래의 표와 같이 분할을 적용하였다.

구분	비율	이미지 수	사용 목적
Train	80%	809장	모델 가중치 학습
Validation	10%	101장	학습 중 성능 확인
Test	10%	102장	최종 분류 성능 평가

본 데이터셋은 친환경 번호판 원본 이미지 수가 부족하여 먼저 데이터 증강을 수행한 뒤 전체 데이터를 학습·검증·테스트 세트로 분할하였다. 이로 인해 동일한 원본에서 생성된 유사 이미지가 서로 다른 데이터 세트에 포함될 가능성이 있으며, 평가 성능이 실제 일반화 성능보다 높게 측정될 수 있다는 한계가 있다.

다만 적용한 증강은 회전, 원근 변환, 위치 이동과 같은 기하학적 변환으로 구성되어 친환경 번호판 분류의 핵심 특징인 파란색 배경과 번호판의 기본 형태는 유지하였다. 따라서 제한된 데이터 환경에서 클래스 불균형을 완화하기 위한 선택으로 활용하였으며, 향후 충분한 원본 데이터가 확보된다면, 원본 이미지를 먼저 학습·검증·테스트 세트로 분할한 뒤 학습 데이터에만 증강을 적용하여 데이터 누수 가능성을 줄일 필요가 있다.

2.2.7 입력 전처리

모든 번호판 이미지는 ResNet18의 입력 크기에 맞추기 위해 224×224 크기로 리사이즈하였다. 이 과정에서는 별도의 패딩을 적용하지 않아 원본 번호판의 가로세로 비율이 일부 변형될 수 있으나, 학습과 추론 단계에 동일한 전처리를 적용하여 입력 형식을 일관되게 유지하였다.

정규화에는 RGB 채널별 평균값 [0.485, 0.456, 0.406]과 표준편차 [0.229, 0.224, 0.225]를 사용하였다. 이를 통해 사전학습 모델이 학습된 조건과 유사한 입력을 제공하여 학습 안정성과 전이학습 효과를 높이하고자 하였다.

3. 모델 설계 및 학습

3.1 YOLOv8 모델 학습

3.1.1 모델 선정 이유

본 프로젝트는 차량과 번호판을 검출한 뒤 차량 5부제 위반 여부를 판정하는 시스템으로, 실제 단속 환경을 고려하여 빠른 이미지 처리가 필요하다. 이에 객체 검출 속도와 정확도의 균형이 우수한 YOLOv8을 선택하였다.

YOLOv8 계열 중에서는 연산량이 적고 추론 속도가 빠른 경량 모델인 YOLOv8n을 사용하였다. 이를 통해 제한된 GPU 환경에서도 CCTV 이미지를 효율적으로 처리하고자 하였다.

3.1.2 모델 구조

YOLOv8n은 Ultralytics에서 개발한 YOLOv8 시리즈 중 가장 경량화된 모델로, Backbone, Neck, Head의 3단계 구조로 이루어져 있다.

구성 요소	세부 내용
Backbone	CSPDarknet - 입력 이미지에서 다중 스케일 특징 추출
Neck	PANet FPN - 다양한 크기의 객체 탐지를 위한 특징 맵 융합
Head	Decoupled Head - 차량 및 번호판 클래스 동시 출력

Backbone은 CCTV 이미지에서 차량과 번호판의 형태 및 질감과 같은 특징을 추출한다. 그리고 Neck은 여러 해상도의 특징 맵을 결합하여 크기가 큰 차량과 상대적으로 작은 번호판을 함께 탐지할 수 있도록 한다. 마지막으로 Head는 각 객체의 Bounding Box 좌표, 클래스 및 신뢰도 점수를 출력한다.

본 프로젝트에서는 차량과 번호판을 각각 car와 plate 클래스로 설정하여 하나의 모델에서 동시에 검출하도록 구성하였다.

3.1.3 학습 데이터 설정

```
dataset/
├── images/
│   ├── train/    # 학습 이미지 4,400장
│   ├── val/      # 검증 이미지 550장
│   └── test/      # 테스트 이미지 550장
├── labels/
│   ├── train/    # 학습 라벨 4,400개
│   ├── val/      # 검증 라벨 550개
│   └── test/      # 테스트 라벨 550개
└── data.yaml
```

YOLOv8 학습을 위해 이미지와 라벨 파일을 Train, Validation, Test 폴더로 구분하여 구성하였다. 각 이미지 파일은 동일한 이름의 YOLO 형식 TXT 라벨 파일과 1:1로 대응한다.

```
path: C:/PyTorch Project/YOLOv8/dataset
train: images/train
val: images/val
test: images/test

nc: 2
names:
  0: car
  1: plate
```

또한 위의 그림과 같이 data.yaml 파일에 데이터셋의 기본 경로와 Train, Validation, Test 경로, 클래스 수 및 클래스 이름을 정의하였다. nc는 클래스 수, names는 클래스 이름을 정의하며, path를 기준으로 train/val/test 경로가 결정된다.

YOLOv8은 학습 시 data.yaml 파일을 참조하여 각 데이터셋의 위치와 클래스 정보를 자동으로 불러온다. 이를 통해 학습 코드에 경로와 클래스 정보를 반복해서 작성하지 않고, 데이터셋 설정을 하나의 파일에서 일관되게 관리할 수 있다.

본 프로젝트의 클래스는 차량을 의미하는 car와 번호판을 의미하는 plate의 두 가지로 구성하였다.

3.1.4 학습 파라미터

YOLOv8n 모델은 사전학습 가중치인 yolov8n.pt를 불러와 차량 및 번호판 데이터셋에 대해 Fine-tuning하였다. 주요 학습 파라미터는 다음과 같이 설정하였다.

항목	설정 값
Epochs	100
Batch Size	16
Image Size	640x640
Optimizer	AdamW
Learning Rate	0.001
Device	GPU (RTX 4070Ti)
Patience	50

Epoch은 5,500장 규모의 데이터셋이 충분히 학습될 수 있도록 최대 100으로 설정하였으며, 검증 성능이 50 epoch 동안 개선되지 않으면 학습을 종료하도록 patience=50을 적용하였다.

Batch Size는 VRAM 용량과 학습 안정성을 고려하여 16으로 설정하였다. Batch Size가 지나치게 작으면 Gradient 추정이 불안정할 수 있고, 지나치게 크면 GPU 메모리 부족이 발생할 수 있으므로 두 조건 사이의 균형을 고려하였다.

입력 이미지 크기는 YOLOv8의 기본 학습 크기인 640×640으로 설정하였다. 이를 통해 서로 다른 해상도의 CCTV 이미지를 동일한 크기로 입력하고, 연산량을 제한하면서 번호판과 같은 소형 객체의 특징을 학습할 수 있도록 하였다.

Optimizer는 AdamW를 사용하였다. AdamW는 가중치 감소를 최적화 과정과 분리하여 적용하므로 과적합 완화에 유리하다. 초기 Learning Rate는 0.001로 설정하여 사전학습된 특징을 크게 훼손하지 않으면서 차량 및 번호판 데이터에 적응하도록 하였다.

3.1.5 학습 코드

```
from ultralytics import YOLO
from pathlib import Path

model = YOLO("yolov8n.pt")

model.train(
    data=r"C:/PyTorch Project/YOLOv8/dataset/data.yaml",
    epochs=100,
    batch=16,
    imgsz=640,
    optimizer="AdamW",
    lr0=0.001,
    device=0,
    project=r"C:/PyTorch Project/YOLOv8/runs",
    name="yolo_train",
    save=True,
    exist_ok=True,
)
```

학습은 Ultralytics 라이브러리의 YOLO 클래스를 활용하여 수행하였다. 먼저 사전학습 가중치인 yolov8n.pt를 불러온 뒤, model.train()에 앞서 설정한 data.yaml 경로와 학습 파라미터를 전달하여 전체 학습을 진행하였다.

학습 과정에서 생성되는 가중치와 성능 지표는 project와 name으로 지정한 경로에 자동으로 저장된다. 또한 exist_ok=True를 설정하여 동일한 저장 경로에서 재학습을 수행하더라도 오류가 발생하지 않도록 하였다.

3.2 ResNet18 모델 학습

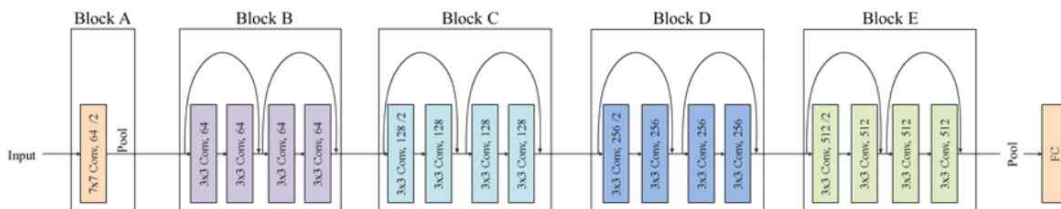
3.2.1 모델 선정 이유

본 프로젝트에서는 친환경 번호판과 일반 번호판을 분류하기 위해 ResNet18을 사용하였다. ResNet18은 Skip Connection을 통해 깊은 신경망에서 발생할 수 있는 기울기 소실 문제를 완화하며, 비교적 적은 연산량으로도 안정적인 이미지 분류 성능을 제공한다.

또한 ImageNet으로 사전학습된 가중치를 활용할 수 있어, 제한된 번호판 데이터에서도 전이학습을 통해 효과적으로 학습할 수 있다는 점을 고려하여 ResNet18을 선정하였다.

3.2.2 모델 구조

ResNet18은 합성곱 계층을 통해 이미지의 특징을 추출하고, Residual Block에 적용된 Skip Connection을 통해 입력 특징을 다음 계층의 출력에 직접 더하는 구조를 사용한다. 이를 통해 네트워크가 깊어질 때 발생할 수 있는 기울기 소실 문제를 완화하고 안정적인 학습이 가능하다.



ResNet18은 초기 합성곱 계층 이후 64, 128, 256, 512개의 채널을 갖는 Residual Block이 각각 2개씩 배치된 2-2-2-2 구조로 구성된다. 각 단계가 진행될수록 특징 맵의 크기는 감소하고 채널 수는 증가하여, 번호판의 색상과 형태와 같은 고수준 특징을 추출한다.

본 프로젝트에서는 ImageNet으로 사전학습된 ResNet18을 사용하고, 기존 1,000개 클래스를 출력하는 마지막 Fully Connected Layer를 친환경 번호판과 일반 번호판을 분류하도록 Linear(512, 2) 구조로 변경하였다. 또한 사전학습된 특징 추출 능력을 유지하기 위해 Backbone의 파라미터는 동결하고, 새롭게 교체한 Fully Connected Layer만 학습하였다.

3.2.3 학습 데이터 설정

```
ResNet18/
├── 번호판 이미지 80,000장/      # 일반 번호판 원본
├── ECO 번호판_합본/            # 친환경 번호판(원본 + 증강 합본)
└── dataset/
    ├── train/
    │   ├── normal
    │   └── eco
    ├── val/
    │   ├── normal
    │   └── eco
    └── test/
        ├── normal
        └── eco
```

ResNet18 학습을 위한 데이터셋 폴더 구조는 위의 그림과 같이 구성하였다. 일반 번호판은 친환경 번호판 사진이 없는 80,000장 원본에서 500장을 무작위 샘플링하여 사용하였다. 친환경 번호판은 원본 128장과 오프라인 증강으로 생성한 384장을 합본하여 총 512장으로 구성하였다.

구성된 데이터는 학습용, 검증용, 테스트용으로 분할한 뒤 각각 train, val, test 폴더에 저장하였다. 각 폴더 내부에는 일반 번호판을 저장하는 normal 폴더와 친환경 번호판을 저장하는 eco 폴더를 생성하였다.

각 폴더 내 이미지 수는 다음과 같다.

구분	일반	친환경	전체
Train	400장	409장	809장
Validation	50장	51장	101장
Test	50장	52장	102장
합계	500장	512장	1,012장

3.2.4 학습 파라미터 설정

ResNet18은 ImageNet으로 사전학습된 IMAGENET1K_V1 가중치를 사용하여 학습을 진행하였다. Backbone 전체 파라미터를 동결(freeze)하고 마지막 FC Layer만 교체하여 학습함으로써, 한정된 번호판 데이터로도 사전학습된 특징 추출 능력을 그대로 활용할 수 있도록 하였다. 주요 학습 파라미터는 다음과 같다.

항목	설정 값
Epochs	30
Batch Size	32
Image Size	224x224
Optimizer	AdamW
Learning Rate	0.001
Loss Function	CrossEntropyLoss
Best 저장 기준	val_acc
Device	GPU (RTX 4070Ti)

Epoch은 특징 추출부를 동결하고 마지막 FC Layer만 학습하는 구조이므로 비교적 빠르게 수렴할 수 있다고 판단하여 30으로 설정하였다. Batch Size는 학습 안정성과 GPU 메모리 효율을 고려하여 32로 설정하였다.

입력 이미지 크기는 ResNet18의 표준 입력 크기인 224×224로 설정하였다. 이를 통해 ImageNet 사전학습 가중치가 학습된 입력 조건과 유사한 형태를 유지하였다.

Optimizer는 AdamW를 사용하였다. AdamW는 가중치 감쇠를 최적화 과정과 분리하여 적용하므로 과적합 완화에 유리하다. Learning Rate는 새롭게 교체한 FC Layer가 빠르게 수렴할 수 있도록 0.001로 설정하였다.

학습 과정에서는 Validation Accuracy가 가장 높게 측정된 시점의 가중치를 Best 모델로 저장하였다. 이는 친환경 번호판과 일반 번호판의 전체 분류 정확도를 기준으로 가장 우수한 모델을 선택하기 위함이다.

3.2.5 학습 코드

```
1 import os
2 import shutil
3 import random
4 import torch
5 import torch.nn as nn
6 from torch.utils.data import DataLoader
7 from torchvision import datasets, transforms, models
8 from sklearn.model_selection import train_test_split
9 from sklearn.metrics import confusion_matrix, classification_report
10 import matplotlib.pyplot as plt
11 import seaborn as sns
12 import numpy as np
13
14 # --- 1. 경로 설정 ---
15 NORMAL_DIR = r"C:/PyTorch Project/ResNet18/번호판 이미지 80,000장"
16 ECO_DIR = r"C:/PyTorch Project/ResNet18/ECO 번호판 합본"
17 DATASET_DIR = r"C:/PyTorch Project/ResNet18/dataset"
18 SAVE_PATH = r"C:/PyTorch Project/ResNet18/resnet18.pth"
19
```

먼저 학습에 필요한 라이브러리를 불러오고, 일반 번호판과 친환경 번호판의 원본 데이터 경로, 학습용 데이터셋 저장 경로, 학습된 ResNet18 가중치 저장 경로를 설정하였다.

```
1 # --- 2. 데이터셋 준비 ---
2 def prepare_dataset():
3     for split in ["train", "val", "test"]:
4         for cls in ["normal", "eco"]:
5             os.makedirs(f"{DATASET_DIR}/{split}/{cls}", exist_ok=True)
6
7     def split_and_copy(src_dir, cls, n_sample=None):
8         files = [f for f in os.listdir(src_dir)
9                  if f.lower().endswith((".jpg", ".png", ".jpeg"))]
10        if n_sample:
11            files = random.sample(files, n_sample)
12        train, temp = train_test_split(files, test_size=0.2, random_state=42)
13        val, test = train_test_split(temp, test_size=0.5, random_state=42)
14        for split, file_list in [("train", train), ("val", val), ("test", test)]:
15            for f in file_list:
16                shutil.copy(os.path.join(src_dir, f),
17                            os.path.join(DATASET_DIR, split, cls, f))
18
19    split_and_copy(NORMAL_DIR, "normal", n_sample=500)
20    split_and_copy(ECO_DIR, "eco", n_sample=None)
21    print("데이터셋 준비 완료")
22
23 prepare_dataset()
```

학습·검증·테스트 데이터로 8:1:1 비율로 분할하여 해당 클래스 폴더에 복사한다. 일반 번호판은 전체 이미지 중 500장을 무작위로 샘플링하며, random_state=42를 사용하여 동일한 분할 결과를 재현할 수 있도록 하였다.

```

1 # — 3. 데이터 변환 —————
2 transform = transforms.Compose([
3     transforms.Resize((224, 224)),
4     transforms.ToTensor(),
5     transforms.Normalize([0.485, 0.456, 0.406],
6                           [0.229, 0.224, 0.225]),
7 ])

```

이후 모든 번호판 이미지를 ResNet18의 입력 크기인 224×224로 조정된 뒤 텐서로 변환한다. 이후 ImageNet 데이터셋의 채널별 평균과 표준편차를 사용해 정규화하여, 사전학습 가중치가 학습된 입력 조건과 유사하게 맞춘다.

```

1 # — 4. 데이터 로더 —————
2 train_ds = datasets.ImageFolder(f"{DATASET_DIR}/train", transform=transform)
3 val_ds   = datasets.ImageFolder(f"{DATASET_DIR}/val",   transform=transform)
4 test_ds  = datasets.ImageFolder(f"{DATASET_DIR}/test",  transform=transform)
5
6 print(f"클래스 인덱스: {train_ds.class_to_idx}")
7
8 train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
9 val_loader   = DataLoader(val_ds,   batch_size=32)
10 test_loader  = DataLoader(test_ds,  batch_size=32)
11
12 print(f"train: {len(train_ds)}장 | val: {len(val_ds)}장 | test: {len(test_ds)}장")

```

ImageFolder를 이용해 학습·검증·테스트 데이터셋을 불러오고, 폴더 이름을 기준으로 클래스 인덱스를 자동 할당한다. 이후 DataLoader를 통해 데이터를 Batch Size 32로 구성하며, 학습 데이터는 매 epoch마다 순서를 섞어 학습 편향을 줄이도록 설정하였다.

```

1 # — 5. 모델 —————
2 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
3 print(f"device: {device}")
4
5 model = models.resnet18(weights="IMAGENET1K_V1")
6
7 for param in model.parameters(): # Backbone freeze
8     param.requires_grad = False
9
10 model.fc = nn.Linear(model.fc.in_features, 2) # FC 교체
11 model = model.to(device)

```

모델은 torchvision.models에서 제공하는 ResNet18을 불러온 후, ImageNet 사전학습 가중치(IMAGENET1K_V1)를 적용하였다. 이후 Backbone 전체 파라미터에 대해 requires_grad=False를 설정하여 학습 중 가중치가 갱신되지 않도록 동결하였다.

마지막으로 기존 FC Layer를 Linear(512→2)로 교체하여 친환경(eco)과 일반(normal) 두 클래스를 출력하도록 구성하였으며, 모델을 GPU 환경으로 이동시켜 학습 속도를 확보하였다.

```

1 # — 6. 학습 설정 —————
2 criterion = nn.CrossEntropyLoss()
3 optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)

```

손실 함수는 다중 클래스 분류에 적합한 CrossEntropyLoss를 사용하였으며, 옵티마이저는 AdamW를 적용하여 FC Layer 파라미터만 학습 대상으로 설정하였다.

```

1 # — 7. 학습 루프 —————
2 EPOCHS      = 30
3 best_val_acc = 0.0
4
5 for epoch in range(EPOCHS):
6     # train
7     model.train()
8     train_loss = 0.0
9     for imgs, labels in train_loader:
10         imgs, labels = imgs.to(device), labels.to(device)
11         optimizer.zero_grad()
12         loss = criterion(model(imgs), labels)
13         loss.backward()
14         optimizer.step()
15         train_loss += loss.item()
16
17     # val
18     model.eval()
19     correct = total = 0
20     with torch.no_grad():
21         for imgs, labels in val_loader:
22             imgs, labels = imgs.to(device), labels.to(device)
23             preds = model(imgs).argmax(dim=1)
24             correct += (preds == labels).sum().item()
25             total += labels.size(0)
26
27     val_acc = correct / total
28
29     print(f"Epoch {epoch+1:02d}/{EPOCHS} | "
30           f"train_loss: {train_loss/len(train_loader):.4f} | "
31           f"val_acc: {val_acc:.4f}")
32
33     if val_acc > best_val_acc:
34         best_val_acc = val_acc
35         torch.save(model.state_dict(), SAVE_PATH)
36         print(f" → best 저장 (val_acc: {val_acc:.4f})")

```

학습은 총 30 epoch 동안 수행하였다. 각 epoch의 Train 단계에서는 순전파를 통해 손실을 계산하고, 역전파를 통해 FC Layer의 파라미터를 갱신하였다. Validation 단계에서는 torch.no_grad()를 적용하여 Gradient 계산을 비활성화한 뒤 검증 정확도를 산출하였다.

매 epoch마다 Validation Accuracy가 이전 최고값보다 높을 경우 해당 시점의 모델 가중치를 저장하였다. 이를 통해 학습 종료 후 검증 성능이 가장 우수한 모델을 최종 모델로 사용할 수 있도록 하였다.

3.3 EasyOCR 모델

EasyOCR은 Python 기반의 오픈소스 OCR 라이브러리로, 80개 이상의 언어를 지원하며 별도의 모델 학습 없이 사전학습된 가중치를 그대로 활용할 수 있다. 본 프로젝트에서는 EasyOCR을 딥러닝 모델이 아닌 번호판 문자 인식을 위한 후처리 도구로 활용하였다.

ResNet18에서 일반 번호판으로 분류된 이미지에 한해 EasyOCR을 적용하여 번호판 문자를 인식한다. 번호판 이미지의 너비가 100픽셀 미만인 경우 3배 업스케일과 샤프닝 필터를 적용하여 인식률을 높인 후 OCR을 수행한다. 인식된 문자열 중 신뢰도 0.3 이상인 텍스트만 사용하고, 해당 문자열에서 숫자만 추출하여 마지막 숫자를 번호판 끝자리로 사용한다. 숫자가 존재하지 않는 경우 None을 반환하여 위반 여부를 판단하지 않도록 하였다.

추출된 끝자리는 아래의 표와 같이 요일별 운행 제한 번호와 비교하여 최종 단속 여부를 판정한다. 주말에는 5부제가 적용되지 않으므로 해당없음으로 처리한다.

요일	운행 제한 끝자리
월요일	1, 6
화요일	2, 7
수요일	3, 8
목요일	4, 9
금요일	5, 0
토요일	해당 없음
일요일	해당 없음

4. 모델 성능 평가

4.1 YOLOv8 모델 성능 평가

4.1.1 최종 성능 지표

학습 완료 후 Test 데이터 550장을 기준으로 최종 성능을 평가하였다. 평가 지표는 객체 탐지 모델의 표준 지표인 mAP@0.5, Precision, Recall, F1-Score를 사용하였다. mAP@0.5는 IoU 임계값 0.5를 기준으로 탐지된 객체의 정밀도와 재현율을 종합한 지표이며, 결과는 아래의 표와 같다.

지표	목표	결과
mAP@0.5	≥ 0.75	0.930
Precision	≥ 0.80	0.938
Recall	≥ 0.75	0.904
F1-Score	≥ 0.75	0.921

설정된 모든 목표치를 초과 달성하였다. 클래스별 세부 결과는 아래의 표와 같다.

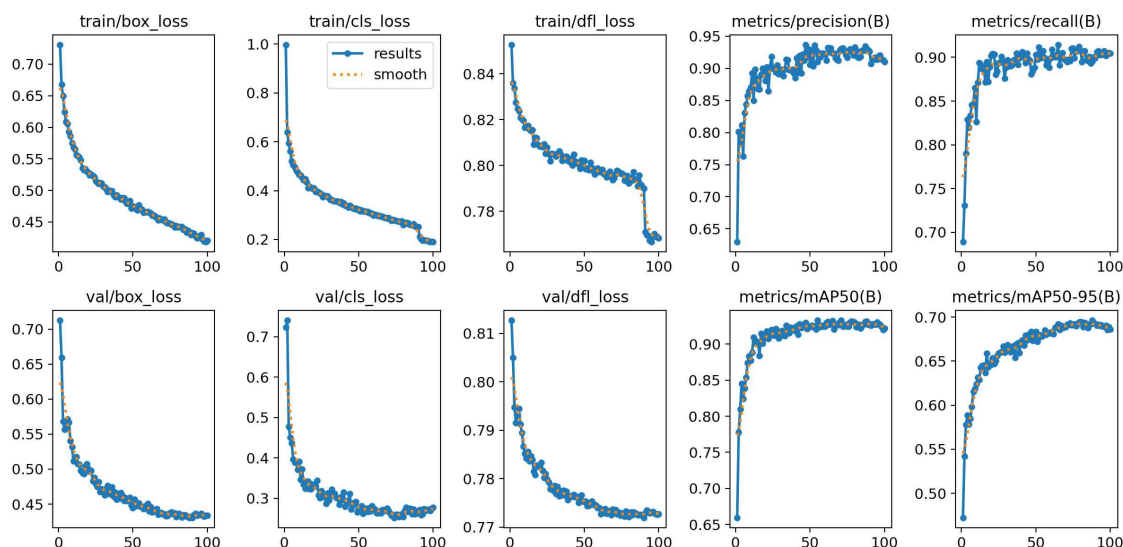
클래스	mAP@0.5	Precision	Recall	F1-Score
car	0.977	0.960	0.964	0.962
plate	0.883	0.916	0.844	0.879
합계	0.930	0.938	0.904	0.921

차량(car) 클래스는 mAP@0.5 0.977, F1-Score 0.962로 매우 높은 탐지 성능을 보였다. 이는 CCTV 이미지에서 차량이 비교적 큰 영역을 차지하고, 형태적 특징이 뚜렷하게 나타나기 때문으로 판단된다.

번호판(plate) 클래스는 전체 이미지에서 차지하는 면적이 작은 소형 객체임에도 mAP@0.5 0.883, F1-Score 0.879를 기록하여 설정된 목표치를 상회하였다. 다만 차량 클래스와 비교했을 때 Recall은 0.844로 상대적으로 낮게 나타났다. 이는 원거리에서 촬영되어 번호판 크기가 매우 작거나, 다른 차량 및 구조물에 의해 일부가 가려진 경우, 또는 촬영 각도와 화질 저하로 번호판의 경계가 불분명한 경우 탐지에 실패한 사례가 존재했기 때문으로 판단된다.

따라서 전체적으로 차량과 번호판 모두 높은 탐지 성능을 확보하였으나, 실제 단속 환경에서 번호판 탐지 안정성을 더욱 높이기 위해서는 입력 해상도 확대, 소형 객체 중심의 데이터 증강, 다양한 거리와 각도의 번호판 데이터 추가 확보가 필요하다.

4.1.2 학습 곡선 분석



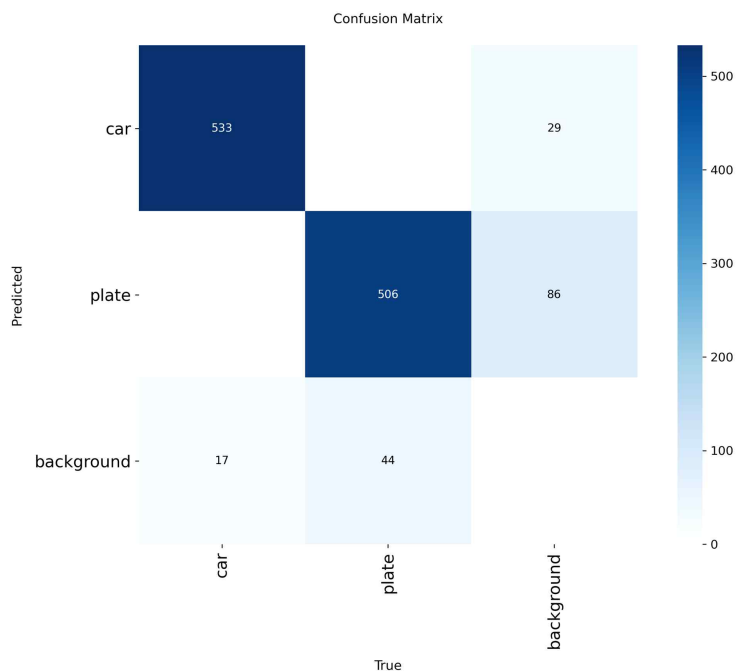
위의 그림은 YOLOv8n 모델의 100 epoch 학습 과정에서 나타난 손실 함수와 성능 지표의 변화를 보여준다.

먼저 train/box_loss, train/cls_loss, train/dfl_loss는 학습 초기에 빠르게 감소한 뒤 점차 완만하게 수렴하는 양상을 보였다. Validation 단계의 val/box_loss, val/cls_loss, val/dfl_loss 역시 Train Loss와 유사한 감소 추세를 나타냈으며, 두 손실 곡선 사이의 차이도 크지 않았다. 이를 통해 학습 데이터에만 지나치게 적합되는 현상 없이 비교적 안정적으로 학습이 진행되었음을 확인할 수 있다.

Precision과 Recall은 학습 초기 약 10 epoch 이내에 빠르게 상승한 후 0.90 이상의 값을 안정적으로 유지하였다. mAP@0.5 역시 초반에 급격히 증가하여 약 20 epoch 이후부터 0.90 이상에서 수렴하였다. 반면 더 엄격한 IoU 기준을 적용하는 mAP@0.5:0.95는 학습 전반에 걸쳐 점진적으로 증가하였으며, 최종적으로 약 0.69에 도달하였다.

종합적으로 Train Loss와 Validation Loss가 함께 감소하고 성능 지표가 안정적으로 수렴한 점을 통해, 본 모델은 뚜렷한 과적합 없이 차량과 번호판의 특징을 효과적으로 학습한 것으로 판단된다.

4.1.3 Confusion Matrix 분석



위의 그림은 Test Set 550장에 대한 YOLOv8의 Confusion Matrix를 나타낸다. 행은 모델이 예측한 클래스, 열은 실제 클래스를 의미하며, background는 실제 객체를 탐지하지 못한 경우 또는 배경을 객체로 잘못 탐지한 경우를 나타낸다.

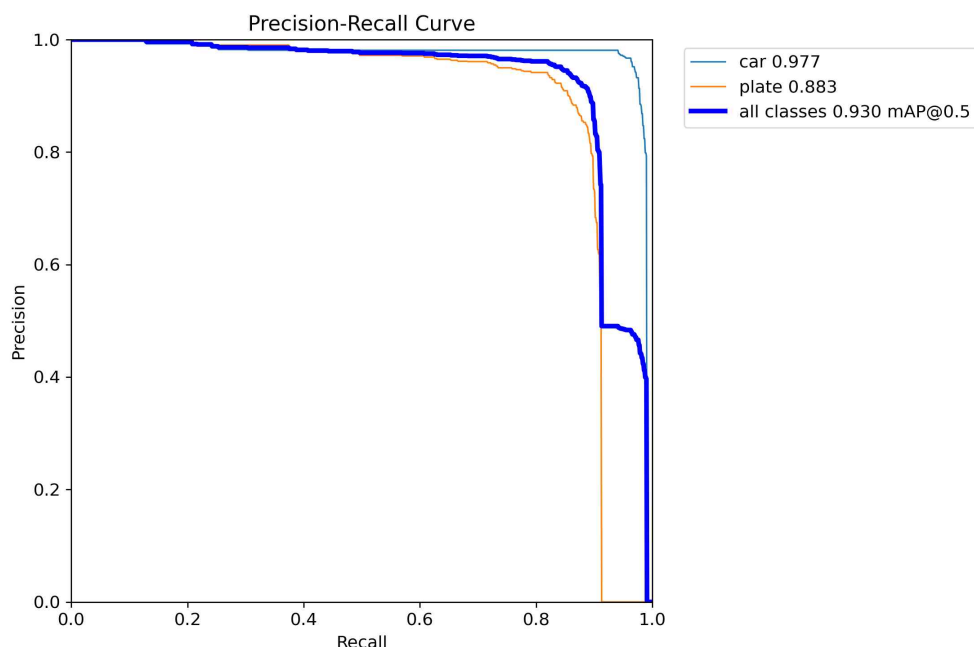
car 클래스

실제 차량 객체 550개 중 533개를 car로 정확하게 탐지하였으며, 17개는 탐지하지 못해 background로 처리되었다. 약 97%의 높은 탐지율을 보였으며, 배경 영역을 차량으로 잘못 탐지한 사례는 29건으로 비교적 적게 나타났다. 이는 차량이 이미지에서 비교적 큰 영역을 차지하고 형태적 특징이 뚜렷하여 모델이 안정적으로 구분한 결과로 판단된다.

plate 클래스

실제 번호판 550개 중 506개를 정확히 탐지하였으며, 44개가 미탐지되었다. 92%의 탐지율을 보였는데, 이는 번호판이 전체 이미지 대비 면적이 작은 소형 객체이기 때문이라고 판단된다. 배경이 plate로 오탐지된 경우는 86건으로 car보다 많았으며, 이는 도로 표지판, 간판 등 번호판과 형태가 유사한 객체가 오탐지된 결과로 판단된다.

4.1.4 Precision-Recall Curve 분석



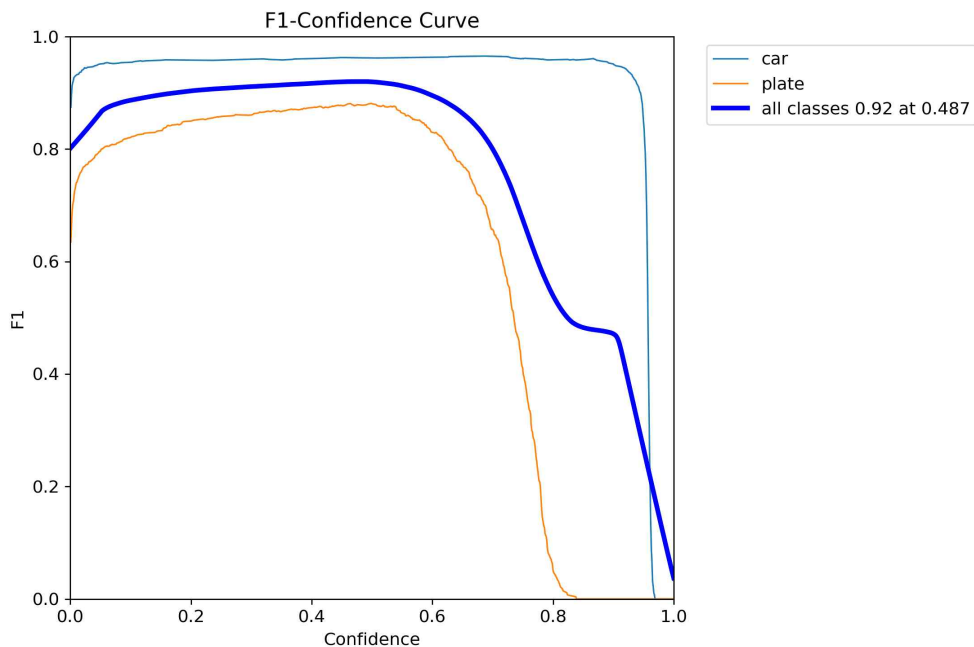
위의 그림은 confidence 임계값 변화에 따른 Precision과 Recall의 관계를 나타낸다. 곡선이 오른쪽 상단에 가까울수록 높은 탐지 성능을 의미하며, 각 클래스 곡선 아래 면적은 AP@0.5, 전체 클래스의 평균은 mAP@0.5에 해당한다.

car 클래스는 AP@0.5 0.977로, Recall이 약 0.9에 도달할 때까지 Precision이 높은 수준으로 유지되어 전반적으로 우수한 차량 탐지 성능을 보였다.

plate 클래스는 AP@0.5 0.883으로 car보다 낮게 나타났다. Recall이 약 0.8까지는 높은 Precision을 유지하지만, 이후 Recall이 증가할수록 Precision이 감소하며 약 0.9 부근에서 급격히 하락하였다. 이는 더 많은 번호판을 탐지하려 할수록 오탐지가 증가하는 소형 객체 탐지의 한계를 보여준다.

전체 클래스의 mAP@0.5는 0.930으로 목표치 0.75를 크게 상회하였으며, 차량과 번호판 모두 전반적으로 높은 탐지 성능을 확보하였다.

4.1.5 F1-Confidence Curve 분석



위의 그림은 confidence 임계값 변화에 따른 F1-Score의 변화를 나타낸다. F1-Score는 Precision과 Recall의 조화 평균으로, 두 지표를 균형 있게 반영한 종합 지표이다.

전체 클래스 기준으로 confidence 0.487에서 최대 F1-Score 0.92를 기록하였다.

car 클래스는 confidence 약 0.1부터 0.9까지의 넓은 구간에서 F1-Score 0.9 이상을 안정적으로 유지하였다. 이를 통해 차량 탐지는 임계값 변화에 크게 영향을 받지 않고 전반적으로 안정적인 성능을 보임을 확인할 수 있다.

plate 클래스는 confidence 약 0.45~0.50 구간에서 가장 높은 F1-Score를 보였으며, 이후 임계값이 증가할수록 성능이 점차 감소하고 약 0.6 이후 하락 폭이 커졌다.

따라서 Precision과 Recall의 균형을 기준으로 하면 confidence 임계값은 약 0.45~0.50으로 설정하는 것이 적절하다. 다만 본 시스템에서는 번호판 미탐지를 줄이는 것이 중요하므로, 일부 오탐지 증가를 감수하고 이보다 낮은 0.30~0.35 수준의 임계값을 적용할 수 있다.

4.1.6 한계점

본 YOLOv8 모델의 학습 및 성능 평가 과정에서는 다음과 같은 한계가 존재한다.

1) 번호판 영역이 보이는 차량만 탐지

학습에 사용된 AI Hub #172 데이터셋은 번호판이 카메라 방향으로 노출된 차량을 중심으로 라벨링되어 있다. 이에 따라 차량의 측면만 보이거나 번호판이 다른 차량 또는 구조물에 가려진 경우에는 차량 탐지가 이루어지지 않는 경향이 있다. 다만 본 시스템은 번호판 인식을 통해 차량 5부제 위반 여부를 판정하는 것을 목적으로 하므로, 번호판을 식별할 수 없는 차량을 판정 대상에서 제외하는 것은 시스템 목적에 부합하는 설계로 볼 수 있다.

2) 원거리 차량 탐지 한계

CCTV 촬영 환경에서는 원거리 차량이 이미지에서 차지하는 영역이 작아 탐지 신뢰도가 낮아질 수 있다. 탐지 신뢰도가 confidence 임계값보다 낮으면 결과에서 제외되며, 임계값을 낮출 경우 원거리 차량의 탐지율은 높아질 수 있지만 배경이나 유사 객체에 대한 오탐지가 증가하는 트레이드오프가 존재한다.

3) 하향식 CCTV 각도에 특화된 도메인

학습 데이터가 대부분 하향식 CCTV 각도에서 촬영된 이미지로 구성되어 있어, 촬영 거리와 각도가 크게 다른 일반 차량 이미지에서는 탐지 성능이 저하될 수 있다. 이는 모델이 학습 데이터의 촬영 환경에 특화된 결과이며, 향후 다양한 각도와 거리에서 촬영된 차량 데이터를 추가로 학습하여 적용 범위를 확장할 필요가 있다.

4.2 ResNet18 모델 성능 평가

4.2.1 최종 성능 지표

학습 완료 후 Test 데이터 102장을 기준으로 최종 성능을 평가하였다. ResNet18은 이진 분류 모델로 친환경 번호판(eco)과 일반 번호판(normal)을 구분하며, 평가 지표는 Accuracy, Precision, Recall, F1-Score를 사용하였다. 결과는 아래의 표와 같다.

지표	목표	결과
Test Accuracy	≥ 0.85	0.951

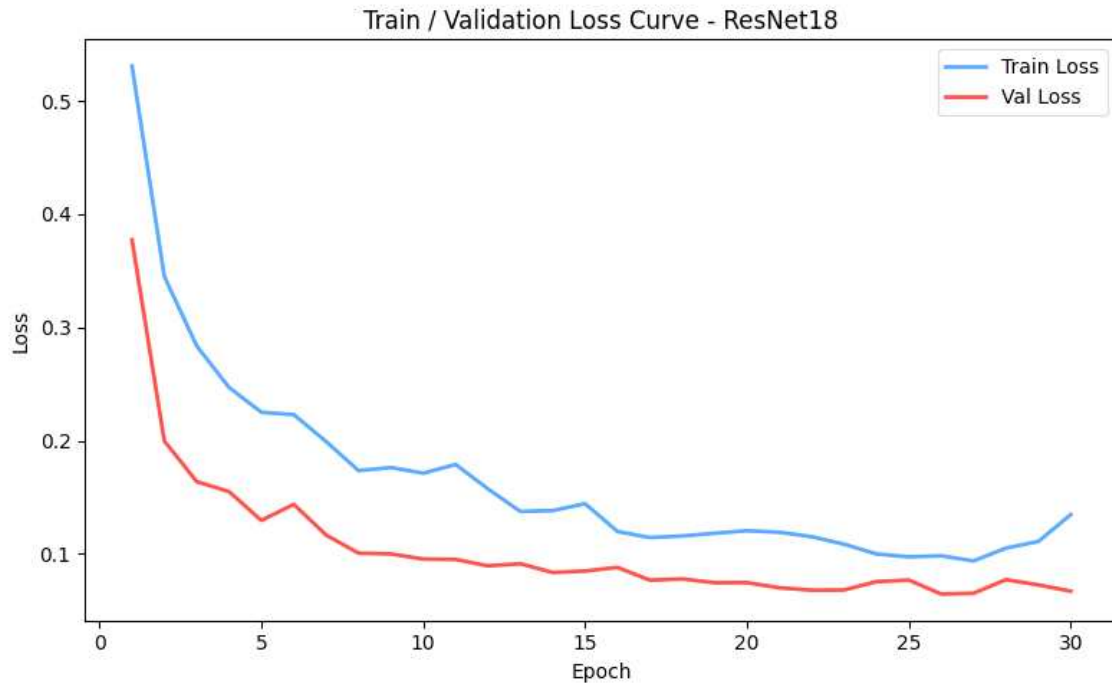
설정된 목표치를 초과 달성하였다. 클래스별 세부 결과는 아래의 표와 같다.

클래스	Precision	Recall	F1-Score	Support
eco	0.927	0.981	0.953	52
normal	0.979	0.920	0.948	50
합계	0.953	0.951	0.951	102

친환경 번호판(eco) 클래스의 Recall은 0.981로 나타났으며, 실제 친환경 번호판 52장 중 51장을 정확하게 분류하였다. 일반 번호판(normal) 클래스는 Precision이 0.979로, 일반 번호판으로 예측한 경우 오분류가 거의 발생하지 않았다.

반면 일반 번호판의 Recall은 0.920으로 상대적으로 낮게 나타났으며, 이는 일반 번호판 4장이 친환경 번호판으로 잘못 분류된 결과이다. 이러한 오류는 실제 단속 대상 차량을 면제 차량으로 판단할 수 있으므로 단속 누락으로 이어질 가능성이 있다.

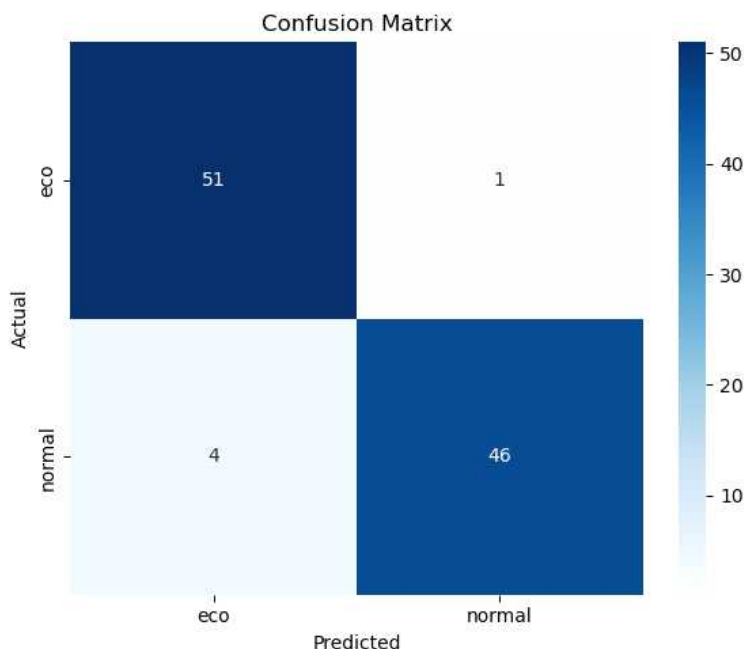
4.2.2 학습 곡선 분석



해당 그래프는 ResNet18 모델의 학습 과정에서 Train Loss와 Validation Loss가 모두 감소하는 양상을 보여준다. 학습 초반에는 손실값이 급격히 감소하여 모델이 변호판 이미지의 주요 특징을 빠르게 학습하고 있음을 확인할 수 있다. 이후 epoch가 증가할수록 손실 감소 폭은 점차 완만해지며, 약 15 epoch 이후부터는 비교적 안정적인 수렴 구간에 들어간다.

Validation Loss 역시 전체적으로 꾸준히 감소하고 있으며, Train Loss와 큰 차이 없이 비슷한 흐름을 보인다. 이는 모델이 학습 데이터에만 과도하게 맞춰지는 것이 아니라 검증 데이터에 대해서도 안정적으로 성능을 유지하고 있음을 의미한다. 따라서 현재 학습 과정에서는 뚜렷한 과적합 현상이 나타나지 않았다고 볼 수 있다.

4.2.3 Confusion Matrix 분석



위의 그림은 test set 102장에 대한 ResNet18의 분류 결과를 나타낸다. 행(Actual)은 실제 클래스, 열(Predicted)은 모델이 예측한 클래스를 의미한다.

eco 클래스

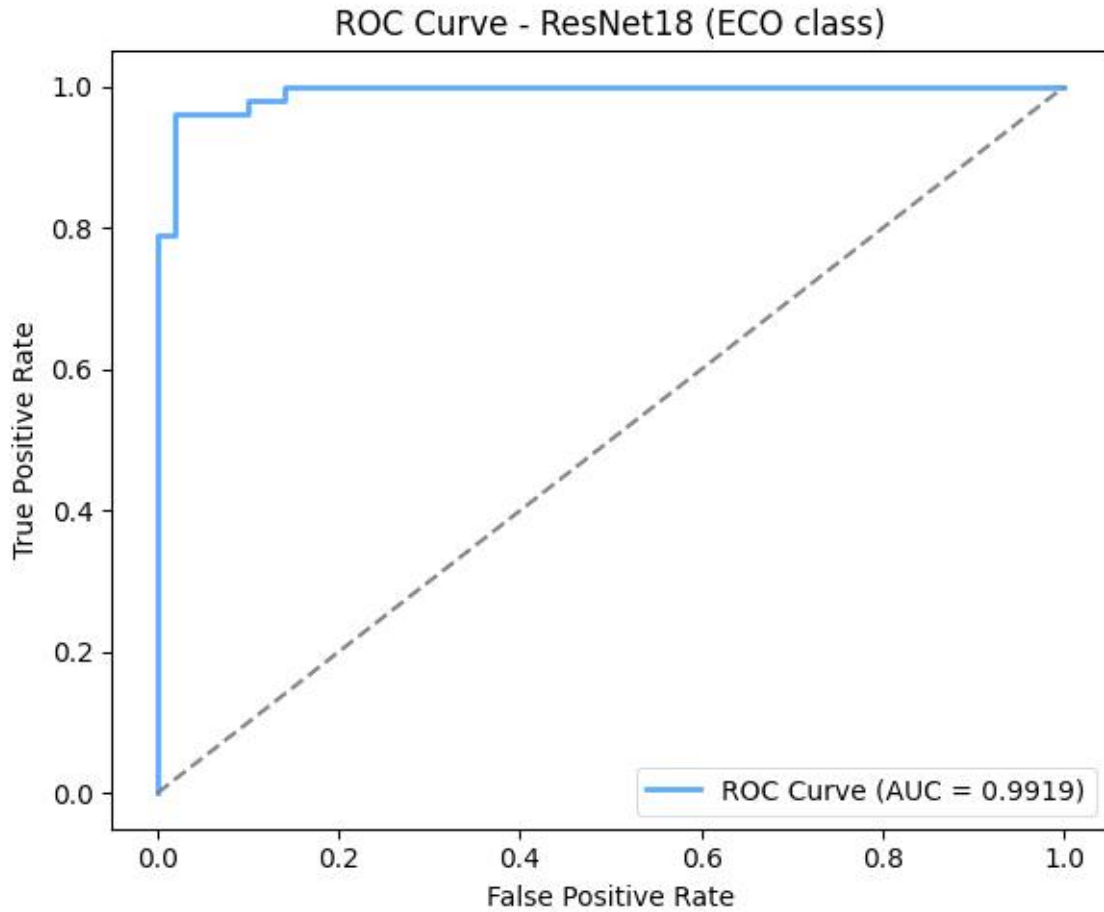
실제 친환경 번호판 52장 중 51장을 정확히 친환경으로 분류하였으며, 1장만 일반 번호판으로 오분류하였다. 이는 친환경 번호판의 파란색 배경이라는 명확한 시각적 특징을 모델이 안정적으로 학습하였음을 의미한다.

normal 클래스

실제 일반 번호판 50장 중 46장을 정확히 일반 번호판으로 분류하였으며, 4장을 친환경 번호판으로 오분류하였다. 이는 일부 일반 번호판 이미지가 조명 또는 촬영 조건에 따라 파란색 계열로 인식된 경우로 판단된다.

전체적으로 Confusion Matrix의 대각선에 값이 집중되어 있어 두 클래스 모두 높은 분류 정확도를 보였다. 이를 통해 제한된 테스트 환경에서는 시스템 적용이 가능한 수준의 성능을 확보한 것으로 판단하였다.

4.2.4 ROC Curve 분석



해당 그래프는 ResNet18 모델의 친환경 번호판 클래스에 대한 ROC Curve를 나타낸 것이다. ROC Curve는 분류 임계값을 변화시키면서 False Positive Rate와 True Positive Rate의 관계를 확인하는 지표로, 곡선이 좌측 상단에 가까울수록 분류 성능이 우수하다고 해석할 수 있다.

그래프에서 ROC 곡선은 대각선 기준선보다 크게 위쪽에 위치하며, 대부분의 구간에서 True Positive Rate가 1에 가깝게 유지된다. 이는 모델이 친환경 번호판을 일반 번호판과 효과적으로 구분하고 있으며, 낮은 오탐률에서도 높은 검출률을 보이고 있음을 의미한다.

또한 AUC 값이 0.9919로 매우 높게 나타났다. AUC는 1에 가까울수록 두 클래스를 잘 구분한다는 의미이므로, 본 모델은 친환경 번호판 클래스에 대해 우수한 분류 성능을 보였다고 할 수 있다. 따라서 ResNet18 모델은 친환경 번호판 여부를 판단하는 단계에서 안정적으로 활용 가능하다고 판단된다.

4.2.5 한계점

본 프로젝트의 ResNet18 학습 및 평가 과정에서 다음과 같은 한계점이 존재한다.

1) 친환경 번호판 원본 데이터 부족

친환경 번호판 원본 이미지는 HSV 색상 필터링과 웹 크롤링을 통해 수집하였으나 총 128장에 그쳤다. 이를 보완하기 위해 RandomRotation, RandomPerspective, RandomAffine 증강을 적용하여 384장을 추가 생성하였으나, 증강 이미지는 동일한 원본에서 파생된 것이므로 실제 데이터 다양성 확보에는 한계가 있다.

2) 증강 후 분할로 인한 데이터 누수 가능성

본 프로젝트에서는 원본과 증강 이미지를 합본한 후 train/val/test로 분할하는 방식을 적용하였다. 이 경우 동일한 원본에서 파생된 이미지가 train 데이터와 Test 데이터에 동시에 포함될 수 있어 평가 신뢰도가 다소 과대 추정될 가능성이 있다. 엄밀한 평가를 위해서는 원본 이미지를 먼저 분할한 후 train set에만 증강을 적용하는 방식이 바람직하며, 이는 향후 개선 방향으로 남겨둔다.

3) Test set 크기 제한

전체 데이터셋 규모가 크지 않아 Test 데이터의 eco 이미지가 52장에 불과하다. 소규모 test set에서는 소수의 오분류만으로도 성능 지표가 크게 변동할 수 있어 평가 결과의 통계적 안정성이 낮을 수 있다.

이러한 한계에도 불구하고 본 프로젝트에서 확보 가능한 데이터 범위 내에서 최선의 방법을 적용하였으며, 설정한 목표치를 초과하여 달성하였다. 향후 충분한 친환경 번호판 데이터셋이 확보된다면 보다 신뢰성 높은 평가와 성능 향상이 가능할 것으로 판단된다.

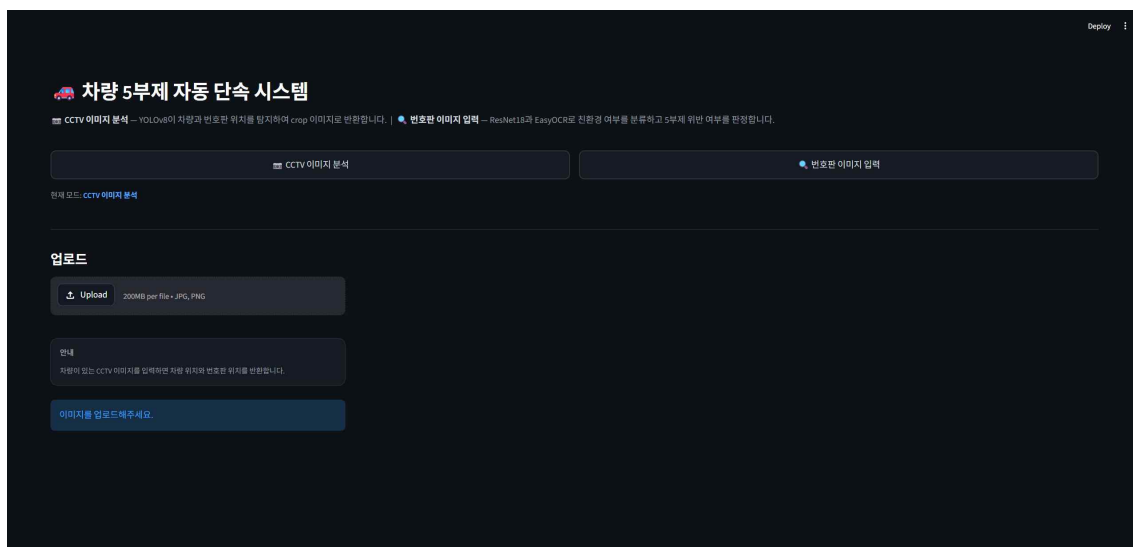
5. 시스템 구현 및 배포

5.1 시스템 구성 및 개발 환경

본 프로젝트의 시스템은 Python 기반의 웹 프레임워크인 Streamlit을 활용하여 구현하였으며, 학습된 YOLOv8, ResNet18, EasyOCR 모델을 통합하여 차량 5부제 단속 기능을 제공한다. 시스템은 로컬 환경에서 개발 및 테스트를 진행하였으며, Hugging Face Spaces를 통해 웹 기반으로 배포하였다.

개발 및 학습 환경은 아래의 표와 같다.

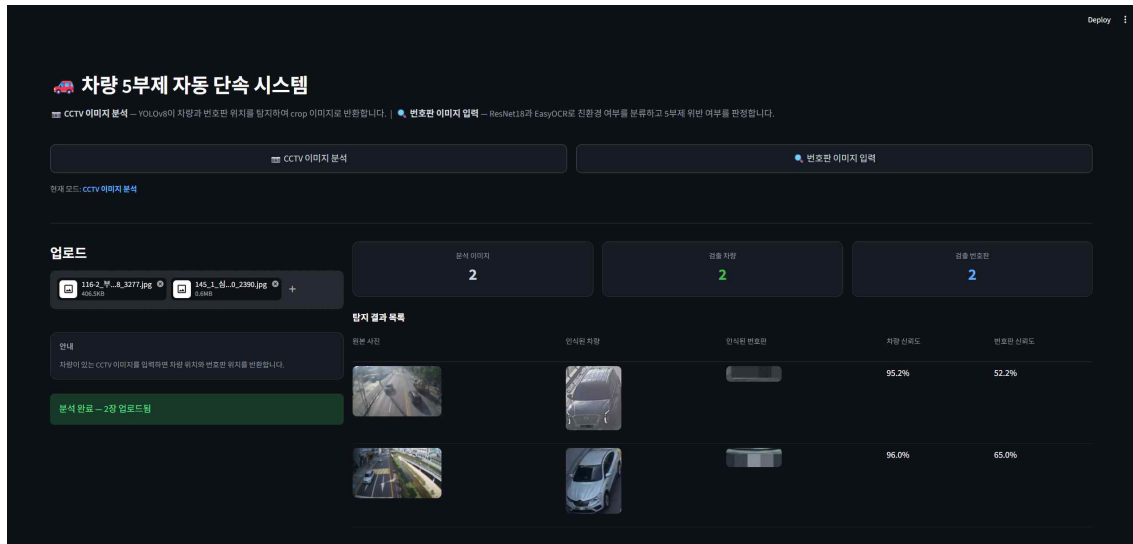
항목	내용
운영체제	Windows 11
CPU	AMD Ryzen7 7800X3D
GPU	NVIDIA GeForce RTX 4070 Ti
RAM	32GB
Python	3.9
딥러닝 프레임워크	PyTorch 2.8.0, CUDA 12.6
주요 라이브러리	Ultralytics, torchvision, EasyOCR, OpenCV, Streamlit
개발 도구	VS Code, Jupyter Notebook, Anaconda
배포 환경	Hugging Face Spaces, Streamlit SDK



Streamlit 웹 인터페이스는 CCTV 이미지 분석 모드와 번호판 이미지 분석 모드로 구성하였다. 사용자는 화면 상단의 버튼을 통해 원하는 기능을 선택하고 여러 장의 이미지를 업로드하여 분석 결과를 확인할 수 있다.

5.2 모드1: CCTV 이미지 분석

모드 1은 CCTV 이미지를 업로드하면 YOLOv8이 차량과 번호판 위치를 탐지하고 검출된 영역을 crop 이미지로 반환하는 객체 검출 기능이다. 여러 장의 이미지를 동시에 업로드할 수 있으며, 각 이미지에 대해 순차적으로 탐지를 수행한다.



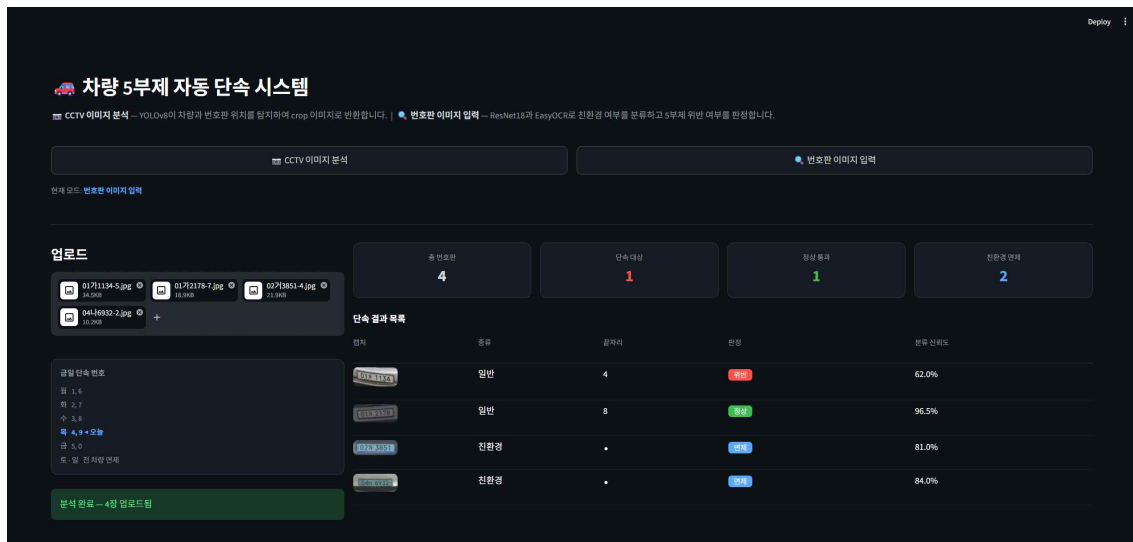
위의 그림은 모드 1의 실제 동작 화면을 나타낸다. CCTV 이미지 2장을 업로드한 결과 총 2대의 차량과 2개의 번호판이 검출되었다. 탐지 결과 목록에는 원본 이미지, 검출된 차량 crop 이미지, 번호판 crop 이미지, 차량 검출 신뢰도 및 번호판 검출 신뢰도가 함께 출력된다.



번호판 crop 이미지는 AI Hub 데이터셋의 특성상 모자이크 처리된 형태로 나타나며, 이는 1.4절에서 설명한 공개 데이터셋의 한계에 해당한다.

5.3 모드2: 번호판 이미지 분석

모드 2는 번호판 이미지를 직접 업로드하면 ResNet18과 EasyOCR을 통해 친환경 여부를 분류하고 5부제 위반 여부를 판정하는 단속 판정 기능이다. 여러 장의 번호판 이미지를 동시에 업로드할 수 있으며, 각 이미지에 대해 순차적으로 분석을 수행한다.



위의 그림은 목요일에 실행한 모드 2의 실제 동작 화면을 나타낸다. 번호판 이미지 4장을 업로드한 결과 단속 대상 1건, 정상 통과 1건, 친환경 면제 2건으로 판정되었다.

영지	종류	끝자리	판정
01가 1134	일반	4	정상
01가 2178	일반	8	정상
02가 3851	친환경	.	면제
04가 6932	친환경	.	면제

단속 결과 목록에는 번호판 이미지, 번호판 종류, 끝자리 숫자, 최종 판정 결과 및 분류 신뢰도가 함께 출력된다. 끝자리가 4인 일반 번호판은 목요일 운행 제한 번호인 4와 9에 해당하여 단속 대상으로 판정되었으며, 끝자리가 6인 일반 번호판은 정상 통과로 판정되었다.

파란색 번호판 2장은 친환경 번호판으로 분류되어 즉시 단속 면제로 처리되었다. 또한 좌측 패널에는 요일별 운행 제한 번호표가 표시되어 현재 요일에 적용되는 제한 번호를 확인할 수 있도록 하였다.

5.4 Streamlit 구현 코드

Streamlit 기반 웹 인터페이스 코드를 작성하였다. 시스템은 모델 로드, 번호판 분류, 문자 인식, 5부제 판정의 순서로 동작하며 주요 코드는 다음과 같다.

1) 모델 로드

@st.cache_resource 데코레이터를 적용하여 YOLOv8, ResNet18, EasyOCR 세 모델을 최초 실행 시 한 번만 로드하고 이후 캐싱하도록 구현하였다.

```
@st.cache_resource
def load_models():
    yolo = YOLO("best.pt")
    resnet = models.resnet18(weights=None)
    resnet.fc = nn.Linear(resnet.fc.in_features, 2)
    resnet.load_state_dict(torch.load("resnet18.pth", map_location=device))
    ocr = easyocr.Reader(['ko', 'en'], gpu=torch.cuda.is_available())
    return yolo, resnet, ocr, device
```

2) YOLOv8 차량·번호판 탐지(모드 1)

confidence 0.35, IoU 0.5를 기준으로 추론을 수행하고 차량 bbox 내부에서 가장 큰 번호판 bbox를 매칭하여 crop 이미지를 반환한다.

```
results = yolo_model.predict(img_rgb, conf=0.35, iou=0.5, verbose=False)
car_boxes = [b for b in boxes if int(b.cls) == 0]
plate_boxes = [b for b in boxes if int(b.cls) == 1]
```

Confidence 0.35는 4.1.5절의 F1-Confidence Curve 분석에서 Precision과 Recall의 균형을 고려한 적정 임계값이 약 0.45~0.50으로 나타났으나, 본 시스템에서는 번호판 미탐지를 줄이는 것이 더 중요하다고 판단하여 이보다 낮게 설정한 값이다. 번호판이 탐지되지 않으면 이후의 친환경 번호판 분류와 문자 인식 과정을 수행할 수 없으므로, 일부 오탐지 증가를 감수하고 최종 Confidence Threshold를 0.35로 설정하였다.

IoU Threshold 0.5는 NMS 과정에서 동일 객체에 대해 중복으로 생성된 bounding box를 제거하기 위한 기준값으로 적용하였다.

3) 친환경 번호판 분류(모드 2)

번호판 이미지를 ResNet18에 입력하여 eco 또는 normal로 분류하고 신뢰도를 반환한다.

```
def classify_plate(crop_rgb):
    x = resnet_transform(pil).unsqueeze(0).to(device)
    with torch.no_grad():
        prob = torch.softmax(resnet_model(x), dim=1)[0]
        pred = prob.argmax().item()
    return ("eco" if pred == 0 else "normal"), float(prob[pred])
```

4) 번호판 끝자리 추출(모드 2)

EasyOCR을 적용하여 신뢰도 0.3 이상인 텍스트에서 숫자만 추출하여 마지막 숫자를 끝자리로 반환한다.

```
def ocr_last_digit(crop_rgb, conf_thr=0.3):
    enhanced = enhance_plate(crop_rgb)
    results = reader.readtext(enhanced)
    filtered = [(t, c) for (_, t, c) in results if c >= conf_thr]
    digits = [c for c in full_text if c.isdigit()]
    return (digits[-1] if digits else None), (full_text if full_text else "-")
```

5) 5부제 위반 판정(모드 2)

추출된 끝자리와 요일별 운행 제한 번호를 비교하여 위반 여부를 판정한다. 요일은 datetime 모듈을 통해 자동으로 파악하며 주말에는 해당없음으로 처리한다.

```
def check_restriction(digit, weekday):
    rules = {0:['1','6'], 1:['2','7'], 2:['3','8'],
             3:['4','9'], 4:['5','0']}
    if weekday >= 5:
        return "해당없음", days[weekday]
    return ("위반" if digit in rules[weekday] else "정상"), days[weekday]
```

5.5 Hugging Face Spaces를 활용한 Streamlit 배포

구현된 시스템은 Hugging Face Spaces를 통해 웹 기반으로 배포하였다. Hugging Face Spaces는 머신러닝 모델을 무료로 배포할 수 있는 플랫폼으로, Streamlit SDK를 지원하여 별도의 서버 구축 없이 간편하게 배포가 가능하다.

 best.pt	파일	설명
 resnet18.pth	best.pt	YOLOv8 학습 완료 가중치
 app.py	resnet18.pth	ResNet18 학습 완료 가중치
 requirements.txt	app.py	Streamlit 웹 인터페이스 및 파이프라인 구현 코드
	requirements.txt	실행에 필요한 라이브러리 목록

배포 시 app.py, best.pt, resnet18.pth, requirements.txt 4개 파일을 업로드하였으며, 파일 업로드 완료 후 자동으로 빌드가 시작되어 수 분 내에 서비스가 활성화된다.

Hugging Face Spaces 배포에 필요한 라이브러리와 버전을 명시한 requirements.txt 파일의 구성은 다음과 같다.

```
streamlit
ultralytics
torch
torchvision
easyocr
opencv-python-headless
matplotlib
```

배포된 시스템은 아래 URL을 통해 접속할 수 있다.

링크: <https://jinwooh30-vehicle-5day-enforcement-system.hf.space>

GitHub 코드 공개

구현에 사용된 전체 코드는 GitHub에 공개하였다. Streamlit 구현 코드(app.py), YOLOv8 학습 코드(YOLOv8.ipynb), ResNet18 학습 코드(ResNet18.ipynb), 학습된 모델 가중치(best.pt, resnet18.pth), 라이브러리 목록(requirements.txt)이 포함되어 있다.

링크: <https://github.com/jinwoo1103/vehicle-5day-enforcement>

5.6 시스템 구현 결과 및 한계

Streamlit 기반 차량 5부제 자동 단속 시스템을 구현하였으며, CCTV 이미지 분석 모드와 번호판 직접 입력 모드 두 가지를 통해 전체 파이프라인의 각 단계가 정상적으로 동작함을 확인하였다. 모드 1에서는 YOLOv8이 CCTV 이미지에서 차량과 번호판을 정상적으로 탐지하고 crop 이미지를 반환하였으며, 모드 2에서는 ResNet18의 친환경 번호판 분류와 EasyOCR의 끝자리 숫자 추출을 통해 5부제 위반 여부 판정까지 전체 흐름이 정상 동작함을 확인하였다.

하지만 이 시스템은 다음과 같은 한계가 존재한다.

1) 번호판 이미지 품질에 따른 OCR 성능 저하

번호판이 흐리거나 조명에 의해 심하게 왜곡된 경우, 또는 촬영 각도가 크게 기울어진 경우에는 EasyOCR의 문자 인식률이 현저히 저하된다. 이에 따라 번호판 끝자리를 정확히 추출하지 못해 판정 불가로 처리되는 경우가 발생한다.

2) 번호판 노출 여부와 거리에 따른 탐지 한계

CCTV 이미지에서 번호판이 카메라 방향으로 노출되지 않은 차량이나 원거리에 위치한 차량은 YOLOv8이 번호판을 탐지하지 못하는 한계가 있다. 이는 학습 데이터의 라벨이 번호판이 카메라 방향으로 노출된 차량을 중심으로 구성되어 있기 때문이다.

3) 무료 배포 환경의 추론 속도 한계

무료 배포 환경에서는 GPU가 제공되지 않아 모델이 CPU 기반으로 동작하므로 추론 속도가 느리다. 또한 Hugging Face Spaces 무료 플랜은 일정 시간 사용하지 않을 경우 절전 모드로 전환되기 때문에, 본 시스템을 실제 실시간 단속에 활용하기 위해서는 GPU를 지원하는 별도의 서버 환경을 구축할 필요가 있다.

따라서 본 프로젝트는 딥러닝 기반 차량 5부제 자동 단속 시스템의 가능성을 검증하는 데 의의가 있으며, 위의 한계점들은 향후 고해상도 CCTV 데이터 확보, GPU 서버 환경 구축, 모자이크 처리가 없는 실제 번호판 데이터셋 수집을 통해 개선할 수 있을 것으로 판단된다.

6. 결론 및 요약

본 프로젝트에서는 YOLOv8, ResNet18, EasyOCR을 결합하여 차량과 번호판을 검출하고, 친환경 차량 여부와 번호판 끝자리를 기반으로 차량 5부제 위반 여부를 자동 판정하는 시스템을 구현하였다. 또한 Streamlit 기반 웹 인터페이스를 제작하고 Hugging Face Spaces에 배포하여 사용자가 이미지 업로드를 통해 단속 결과를 확인할 수 있도록 하였다.

YOLOv8은 테스트 데이터에서 mAP@0.5 0.930, Precision 0.938, Recall 0.904를 기록하여 차량과 번호판을 안정적으로 검출하였다. ResNet18 역시 일반 번호판과 친환경 번호판을 높은 정확도로 분류하였으며, 이를 통해 친환경 차량의 단속 면제 여부를 자동으로 판정할 수 있음을 확인하였다.

다만 공개 CCTV 데이터의 번호판 모자이크로 인해 전체 파이프라인을 동일한 환경에서 검증하기 어려웠으며, 번호판의 해상도, 조명, 촬영 각도와 차량의 거리에 따라 탐지 및 문자 인식 성능이 저하되는 한계가 있었다. 또한 무료 배포 환경에서는 GPU를 사용할 수 없어 실제 실시간 단속 시스템에 적용하기에는 추론 속도가 충분하지 않았다.

향후에는 실제 CCTV 환경에서 수집한 고해상도 번호판 데이터를 추가하고, 원본 데이터를 먼저 분할한 뒤 학습 데이터에만 증강을 적용하여 모델의 일반화 성능을 더욱 신뢰성 있게 평가할 필요가 있다. 또한 번호판 기울기 보정, 초해상도 복원 및 OCR 전처리 기법을 적용하여 문자 인식률을 개선하고, 다양한 촬영 거리와 각도의 데이터를 추가 학습하여 번호판 탐지 성능을 향상할 수 있다. 최종적으로는 GPU 기반 서버와 영상 스트리밍 처리 기능을 구축하여 실시간 CCTV 영상에 적용 가능한 차량 5부제 자동 단속 시스템으로 확장할 수 있다.

7. 참고 문헌

1. 데이터셋 및 이미지 출처

[1] AI Hub. 「차량 및 번호판 인식용 데이터셋」.

<https://aihub.or.kr/aihubdata/data/view.do?currMenu=115&topMenu=100&aihubDataSe=dat a&dataSetSn=172>

[2] PyTorch. *ResNet18 Model Documentation*.

<https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html> /보 고서의 ResNet18 구조 설명 및 이미지 출처로 활용하였다.

2. 참고문헌

[3] Harris, C. R. et al. (2020). "Array programming with NumPy." *Nature*, 585, 357–362.

[4] Jocher, G. et al. (2023). *Ultralytics YOLOv8*.

<https://github.com/ultralytics/ultralytics>

[5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[6] JaidedAI. (2020). *EasyOCR*.

<https://github.com/JaidedAI/EasyOCR>

[7] Paszke, A. et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in Neural Information Processing Systems*, 32.

[8] OpenCV. *Open Source Computer Vision Library*.

<https://opencv.org/>

[9] Streamlit. *The Fastest Way to Build and Share Data Apps*.

<https://streamlit.io/>

[10] Hugging Face. *Hugging Face Spaces Documentation*.

<https://huggingface.co/docs/hub/spaces>

3. 문서 제작 도구

[11] diagrams.net. *Diagramming and Flowchart Software*.

<https://www.diagrams.net/> 시스템 구성도 및 전체 파이프라인 다이어그램 제작에 활용하였다.

[12] Carbon. *Create and Share Beautiful Images of Source Code*.

<https://carbon.now.sh/> 보고서에 삽입한 소스 코드 이미지 제작에 활용하였다.

4. 생성형 AI 활용 내역

본 프로젝트의 코드 오류 수정, 구현 방법 검토 및 보고서 문장 교정 과정에서 ChatGPT(OpenAI), Gemini(Google), Claude(Anthropic)를 보조 도구로 활용하였다. 생성형 AI가 제시한 내용은 프로젝트의 목적, 실제 코드 및 실행 결과와 일치하는지 직접 검토하고 수정한 후 사용하였다.